

ESPRIT

École Supérieure Privée d'Ingénierie et de Technologies
Data Science Engineering Department

SUMMER INTERNSHIP REPORT

Academic year 2025 – 2026

QuizBot

*An intelligent teaching assistant that generates and grades
self-assessment quizzes with a local LLM and Retrieval-Augmented Generation*

Presented by: **Rayen Mannai**

4th-year Data Science Engineering student

Host organisation: **Esprit Tech**

Internship supervisor: **Mrs. Amani Ayeb**

Acknowledgements

Before presenting this work, I would like to thank the people who made this internship possible and helped me along the way.

First, I want to thank my supervisor at Esprit Tech, **Mrs. Amani Ayeb**, for trusting me with this subject, for the time she gave me during our follow-up meetings and for her remarks, which often pushed me to question a solution that "worked" but was not good enough yet. Her insistence on measuring things instead of judging them by eye changed the second half of this project.

I also thank the whole Esprit Tech team for welcoming me and for the practical working conditions during the internship period.

My thanks also go to the teaching staff of ESPRIT, and in particular to the teachers of the Data Science track, whose courses on machine learning, natural language processing and software engineering I relied on constantly during these weeks.

Finally, I thank the members of the jury for agreeing to evaluate this work.

Abstract

Writing good self-assessment quizzes takes teachers a lot of time, and as a result students rarely get enough material to practise on. This report presents **QuizBot**, a web application I designed and developed during my summer internship at Esprit Tech. A teacher uploads a course (PDF or PowerPoint), and the application generates a quiz made of multiple-choice and open questions, which students can then take and get corrected instantly.

The system is built around a Retrieval-Augmented Generation (RAG) pipeline: the course is cleaned, split into overlapping chunks, embedded with a sentence-transformers model and indexed in ChromaDB. At generation time, a query derived from the document itself retrieves candidate passages, and a Maximal Marginal Relevance (MMR) step selects passages that are both relevant and diverse. Questions are then written by an open-source language model (Qwen2.5-Instruct) that runs **entirely on a laptop GPU** thanks to 4-bit quantization, so no course content is sent to an external API. Because a small local model makes mistakes, I added two layers of control: deterministic quality gates (grounding of the cited excerpt, prompt-artifact detection, degenerate choices, internal contradictions) and an optional verification agent that re-reads every question, rejects the weak ones and regenerates them with the rejection reason.

The application has a FastAPI backend secured with JWT and role-based access, a Streamlit interface with a teacher space and a student space, PDF/JSON export, a teacher gradebook with per-question analysis (success rate, discrimination, Cronbach's alpha) and a "semantic map" that projects the course in 2D and overlays students' results. Open answers are graded like a human grader would, with three tiers (right, partial, wrong): clear cases are decided by rules and embedding similarity, and the uncertain ones by a grading agent. On a benchmark of 48 labelled answers, this reduced the mean grading error from 0.32 to 0.12 points without counting any wrong answer as right. The project is covered by 150 automated tests. A retrospective replay of the quality gates on every quiz stored during development shows that 15 of the 55 stored questions would now be rejected, all of them generated before the gates existed.

Keywords: RAG, large language models, question generation, Qwen2.5, quantization, ChromaDB, MMR, LLM-as-a-judge, FastAPI, Streamlit, educational technology.

Résumé

La rédaction de questionnaires d'auto-évaluation est une tâche chronophage pour les enseignants. Ce rapport présente **QuizBot**, une application web réalisée lors de mon stage d'été chez Esprit Tech, qui génère automatiquement des QCM et des questions ouvertes à partir d'un support de cours (PDF ou PPTX). Le système repose sur un pipeline RAG (extraction, nettoyage, découpage, vectorisation avec sentence-transformers, indexation ChromaDB, sélection MMR) et sur un modèle de langage open-source, Qwen2.5-Instruct, exécuté localement sur GPU grâce à une quantification 4 bits. La qualité des questions est contrôlée par des portes de qualité déterministes et par un agent de vérification qui relit, rejette et régénère les questions faibles. L'application comprend une API FastAPI sécurisée par JWT, une interface Streamlit (espace enseignant et espace étudiant), une correction automatique, un export PDF/JSON et une carte sémantique du cours. Les questions ouvertes sont corrigées selon un barème à trois paliers (juste, partiel, faux), avec un agent de correction pour les cas incertains, et l'enseignant dispose d'un carnet de notes avec une analyse des questions. L'application est couverte par 150 tests automatisés.

Mots-clés : RAG, LLM, génération de questions, quantification, ChromaDB, MMR, agent de vérification, FastAPI, Streamlit.

Table of Contents

General Introduction.....	1
Chapter 1: Project Context.....	2
Introduction.....	2
1.1 Host Organisation: Esprit Tech.....	2
1.2 Problem Statement.....	2
1.3 Study of Existing Solutions.....	3
1.4 Proposed Solution.....	3
1.5 Working Methodology.....	4
Conclusion.....	4
Chapter 2: Requirements Analysis and Specification.....	5
Introduction.....	5
2.1 Actors.....	5
2.2 Functional Requirements.....	5
2.3 Non-Functional Requirements.....	6
2.4 Use Case Diagram.....	6
2.5 Textual Description of Main Use Cases.....	7
2.6 Project Planning.....	7
Conclusion.....	8
Chapter 3: Theoretical Background.....	9
Introduction.....	9
3.1 Large Language Models.....	9
3.2 Retrieval-Augmented Generation (RAG).....	9
3.3 Chunking.....	10
3.4 Sentence Embeddings.....	10
3.5 Vector Databases and Approximate Nearest Neighbour Search.....	10
3.6 Maximal Marginal Relevance (MMR).....	10
3.7 Quantization of Language Models.....	11
3.8 Using an LLM to Verify an LLM.....	11
3.9 Automatic Question Generation in Education.....	11
3.10 Dimensionality Reduction for Visualisation.....	11
3.11 Item Analysis (Classical Test Theory).....	11
Conclusion.....	12
Chapter 4: Design of the Solution.....	13
Introduction.....	13
4.1 Global Architecture.....	13
4.2 Design of the RAG Pipeline.....	13
4.3 Design of the Verification Agent.....	14
4.4 Data Model.....	15
4.5 Dynamic View: Sequence Diagrams.....	15
4.6 REST API Design.....	17
Conclusion.....	17
Chapter 5: Implementation.....	18
Introduction.....	18
5.1 Working Environment.....	18
5.2 Technology Choices.....	18

5.3 Document Ingestion.....	19
5.4 Indexing and Retrieval.....	20
5.5 The LLM Abstraction Layer.....	20
5.6 Prompt Engineering.....	21
5.7 Parsing and Salvaging the Model Output.....	22
5.8 Programmatic Quality Gates.....	22
5.9 Verification Agent.....	23
5.10 Automatic Grading.....	23
5.11 Teacher Gradebook and Item Analysis.....	24
5.12 Semantic Map of a Course.....	26
5.13 Export.....	26
5.14 Authentication and Security.....	27
5.15 User Interface.....	27
5.16 Deployment.....	31
Conclusion.....	31
Chapter 6: Tests, Evaluation and Results.....	33
Introduction.....	33
6.1 Automated Tests.....	33
6.2 Ingestion on Real Courses.....	34
6.3 Quality Gates: Retrospective Replay.....	34
6.4 Verification Agent.....	35
6.5 Grading of Open Answers.....	36
6.6 Calibration of MMR.....	36
6.7 Qualitative Analysis of Generated Questions.....	37
6.8 Toward a Quantitative Evaluation.....	38
Conclusion.....	38
General Conclusion and Perspectives.....	39
Perspectives.....	39
References.....	41
Appendix A: Main Configuration Variables.....	43
Appendix B: Project Structure.....	44

List of Figures

Figure 1: Global use case diagram.....	6
Figure 2: Project planning (Gantt chart, in weeks).....	8
Figure 3: Global architecture of QuizBot.....	13
Figure 4: QuizBot RAG pipeline, from upload to stored quiz.....	13
Figure 5: Generate, verify, regenerate loop of the verification agent.....	14
Figure 6: Class diagram of the main entities.....	15
Figure 7: Sequence diagram: generating a quiz.....	16
Figure 8: Sequence diagram: submitting and grading a quiz.....	16
Figure 9: Teacher space: gradebook of a quiz (real results of the Python quiz).....	25
Figure 10: Teacher space: per-question analysis (indicative here, with a single copy).....	26
Figure 11: Home and login screen.....	27
Figure 12: Teacher space: uploading a course and list of indexed documents.....	28
Figure 13: Teacher space: quiz configuration, with the option to enable the verification agent.....	28
Figure 14: Teacher space: library of generated quizzes (publish, export PDF/JSON).....	29
Figure 15: Student space: taking a published quiz (MCQs and open questions).....	30
Figure 16: Student space: score and statistics by theme and question type.....	30
Figure 17: First page of a quiz exported as PDF (generated by Qwen2.5 on the AI introduction course)	31
Figure 18: Distribution of the 150 automated tests by module.....	33
Figure 19: Stored quizzes, in chronological order, re-checked by the current quality gates.....	34
Figure 20: Outcome of the verification agent on 21 candidate questions.....	35
Figure 21: Sections covered by the 4 selected passages as a function of λ (synthetic course, 200 draws).....	37

List of Tables

Table 1: Comparison of existing quiz solutions.....	3
Table 2: Functional requirements.....	5
Table 3: Description of the "Generate a quiz" use case.....	7
Table 4: Description of the "Take a quiz" use case.....	7
Table 5: Sprint backlog summary.....	8
Table 6: Main endpoints of the QuizBot API.....	17
Table 7: Working environment.....	18
Table 8: Technology choices and justification.....	18
Table 9: Programmatic quality gates.....	22
Table 10: Effect of the ingestion pipeline on the four test courses.....	34
Table 11: Open-answer grading on 48 labelled answers.....	36
Table 12: Evolution of the generated questions during the internship.....	37

List of Acronyms

Acronym	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
GPU / VRAM	Graphics Processing Unit / Video RAM
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbour index)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LMS / ENT	Learning Management System / Espace Numérique de Travail
MCQ (QCM)	Multiple-Choice Question (Question à Choix Multiples)
MMR	Maximal Marginal Relevance
NF4	4-bit NormalFloat quantization data type
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
PCA (ACP)	Principal Component Analysis
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
t-SNE	t-distributed Stochastic Neighbor Embedding
TF	Term Frequency
CSV	Comma-Separated Values
UML	Unified Modeling Language

General Introduction

Self-assessment is one of the most effective ways to learn: answering questions about a course, getting feedback and going back to what was not understood helps students much more than re-reading slides. The problem is that someone has to write those questions. For a teacher who already prepares lectures, labs and exams, writing a good set of multiple-choice questions for every chapter, with credible wrong answers and a clear explanation, is a heavy extra workload. In practice, students usually end up with very few practice quizzes, often the same ones from year to year.

Large language models (LLMs) changed what is possible here. A model like GPT-4 or Qwen can write a reasonable question from a paragraph in a few seconds. But using an LLM "as is" raises real issues for an educational tool: the model can invent facts that are not in the course (hallucination), it tends to copy sentences instead of testing understanding, and sending course material to an external API is not always acceptable for an institution. The quality of the output also depends heavily on which parts of the course the model actually sees.

This is the context of my summer internship at **Esprit Tech**, carried out as a fourth-year Data Science engineering student at ESPRIT. The subject was to design and build **QuizBot**, an intelligent teaching assistant that generates quizzes automatically from course material using the Retrieval-Augmented Generation (RAG) approach. From the start, I set myself an additional constraint: the language model had to run **locally**, on my own laptop GPU, instead of calling a paid external API. I wanted the project to be a real data science exercise (choosing and quantizing a model, controlling retrieval, measuring quality) and not just a wrapper around someone else's service.

Very quickly, the main difficulty turned out not to be "making it work" but making the questions **good**: grounded in the course, conceptual rather than copied, with plausible distractors and a correct answer. A large part of the work described in this report is therefore about quality control: better retrieval (a document-derived query and MMR), better prompting, deterministic quality gates and an agent that verifies and regenerates questions.

This report is organised as follows. **Chapter 1** presents the host organisation, the problem and the methodology I followed. **Chapter 2** details the requirements and the planning. **Chapter 3** introduces the theoretical background (LLMs, RAG, embeddings, vector search, MMR, quantization, LLM-based verification, test item analysis). **Chapter 4** describes the design of the solution. **Chapter 5** goes through the implementation, module by module, with screenshots of the application. **Chapter 6** presents the tests and the evaluation results, including the problems I observed in the generated questions. The report ends with a general conclusion and the perspectives of the project.

CHAPTER 1

Project Context

Introduction

This first chapter places the project in its context. I start by presenting the host organisation, then I describe the problem QuizBot addresses, review existing solutions, and explain the solution I proposed and the working methodology.

1.1 Host Organisation: Esprit Tech

I carried out this internship at **Esprit Tech**, in Tunis. Working in an environment close to engineering education gave this project a very concrete target user: teachers who prepare courses every week, and students who need to practise on them. The courses I used to test the application throughout the internship (an introduction to artificial intelligence, a Python course, a machine learning methods course and an HTML course) are typical of what an engineering student receives as slides or PDF handouts.

During the internship I worked under the supervision of **Mrs. Amani Ayeb**. The project followed a technical specification ("cahier des charges") describing the expected features: document upload, a complete RAG pipeline, LLM-based generation of MCQs and open questions, a student self-assessment module with instant feedback, PDF/JSON export and a short user guide for teachers.

1.2 Problem Statement

Writing quiz questions manually has three main limits:

- **Time.** A good MCQ needs a clear stem, one correct answer and three wrong answers that are plausible enough to make the student think. Writing ten of these per chapter takes hours.
- **Frequency.** Because it is expensive, practice material is rare and rarely updated when the course changes.
- **Feedback.** A list of questions without explanations or links to the course does not help a student understand why an answer is wrong.

Generating questions with an LLM solves the time problem but creates new ones. A model that only receives a vague instruction ("write questions about this course") will produce generic questions, may invent facts and often just turns a sentence of the course into a question whose answer is the same sentence. The real question of this project is therefore:

How can we automatically generate quiz questions that stay faithful to a given course, test understanding rather than memorisation, and remain reliable when the language model is a small open-source model running on modest hardware?

1.3 Study of Existing Solutions

Before designing anything, I looked at the tools teachers and students already use for quizzes. Table 1 summarises the comparison.

Table 1: Comparison of existing quiz solutions

Solution	What it does well	Limits for our need
Moodle quiz module	Complete quiz engine, integrated with the LMS, many question types	Questions must be written by hand; no generation from a course
Kahoot!, Quizizz	Engaging, live classroom quizzes	Manual authoring; AI features are external and paid, content is sent to their servers
Quizlet	Flashcards and practice modes for students	Mostly memorisation (term / definition), not conceptual questions
General chatbots (ChatGPT, etc.)	Can write questions from pasted text	No grounding guarantee, no traceability, no student space or grading, course content leaves the institution
QuizBot (this project)	Generation grounded in the uploaded course, quality control, grading and statistics, runs locally	Quality bounded by a small local model; scanned PDFs not supported yet

The comparison shows a gap between quiz platforms, which are good at delivering quizzes but rely on manual authoring, and general-purpose chatbots, which can generate text but give no guarantee that the questions come from the course and offer nothing for grading or follow-up. QuizBot aims to sit in the middle: generation that is traceable to the course, plus the workflow around it.

1.4 Proposed Solution

QuizBot is a web application with two spaces. In the **teacher space**, a teacher uploads a PDF or PPTX course, configures a quiz (number of questions, type, difficulty, optional themes, optional verification agent), reviews the generated questions, publishes the quiz, exports it, and follows the results of the class in a gradebook. In the **student space**, a student picks a published quiz, answers it and immediately gets a score, the correction of each question with an explanation, and statistics by theme and question type.

The core of the system is a RAG pipeline: instead of asking the model to write about "the course" in general, the application retrieves the most relevant and diverse passages of the uploaded document and asks the model to write questions **strictly from these passages**, citing a verbatim excerpt as proof. Every generated question is then checked by deterministic gates, and optionally by an agent that verifies and regenerates it.

Three choices define the project:

- **A local model instead of an external API.** I chose to run Qwen2.5-Instruct on my laptop GPU (NVIDIA RTX 3060, 6 GB of VRAM) using 4-bit quantization. This keeps course content on the machine, has no per-request cost, and forced me to deal with the real behaviour of a small model.

- **Grading like a teacher, not like a similarity score.** Open answers get 1, 0.5 or 0 point; only the uncertain cases are sent to a grading agent, and anything it cannot settle is left as a provisional grade for the teacher.
- **An agentic verification loop instead of training a model.** I did not have a labelled dataset of good and bad questions, and fine-tuning a model on a laptop was not realistic in the internship time. Making the system check and correct its own output was a more practical way to improve quality.

1.5 Working Methodology

I followed an iterative and incremental approach inspired by Scrum, organised in short sprints of roughly one week. Each sprint ended with a working version of the application that I could demonstrate during follow-up meetings with my supervisor. This was important for this kind of project, because many design decisions (chunk size, prompt wording, retrieval strategy, gates) could only be made after looking at real generated questions.

In practice, each sprint followed the same loop: implement a feature, write the corresponding automated tests, generate quizzes on real course documents, read the questions critically, and turn the problems I found into the next tasks. The planning is detailed in Chapter 2.

Conclusion

This chapter presented the host organisation, the problem of producing good practice quizzes, the limits of existing tools and the solution I proposed. The next chapter specifies the requirements of QuizBot in detail.

CHAPTER 2

Requirements Analysis and Specification

Introduction

This chapter specifies what QuizBot must do. I first identify the actors, then list the functional and non-functional requirements, model them with a use case diagram and textual descriptions, and finally present the project planning.

2.1 Actors

- **Teacher (professeur):** uploads course material, generates, reviews, publishes and exports quizzes, follows the results in a gradebook, and consults the semantic map of a course with the class performance.
- **Student (étudiant):** takes published quizzes and consults the correction and statistics.
- **Language model (secondary actor):** the local LLM that writes and verifies questions and settles uncertain open answers. It is not a human user, but the system depends on it and its behaviour shapes many requirements.

2.2 Functional Requirements

Table 2: Functional requirements

ID	Requirement	Actor
FR1	Create an account with a role (teacher or student) and log in; teacher sign-up can be protected by a code	All
FR2	Upload a course in PDF or PPTX format; the text is extracted, cleaned, chunked and indexed automatically	Teacher
FR3	Configure a quiz: title, number of questions (1–20), type (MCQ, open, mixed), difficulty, optional themes	Teacher
FR4	Generate the quiz with the RAG pipeline; optionally enable the verification agent	Teacher
FR5	Preview each question with its answer, explanation and source excerpt	Teacher
FR6	Publish a quiz so that students can see it	Teacher
FR7	Export a quiz as PDF or JSON, with or without answers	Teacher
FR8	Visualise the semantic map of a course and overlay students' results	Teacher
FR9	List published quizzes and answer one	Student
FR10	Get an immediate score, per-question correction with explanation, and statistics by theme and type	Student
FR11	Consult the gradebook of a quiz: one row per submission, class statistics, CSV export, detailed copies and a per-question analysis	Teacher

2.3 Non-Functional Requirements

- **Faithfulness:** questions must be based only on the uploaded course; each question must carry a verbatim excerpt that proves where it comes from.
- **Privacy and cost:** course content should not have to leave the machine; the default production provider is a local model, with no per-request cost.
- **Hardware constraint:** the whole stack must run on a laptop with a 6 GB GPU.
- **Security:** passwords hashed, stateless authentication with tokens, role-based access on every endpoint; a student must never be able to read answers before submitting, even by calling the API directly.
- **Robustness:** a malformed or truncated model output must not crash the generation; an unavailable verifier must not block the teacher.
- **Portability:** the LLM provider, the embedding model and the database must be changeable through configuration (.env) without code changes.
- **Testability:** the pipeline must be testable without a GPU or an API key (mock provider, mocked heavy dependencies).
- **Usability:** a clear interface in French, adapted to non-technical teachers, with a short user guide.

2.4 Use Case Diagram

Figure 1 shows the global use case diagram. Authentication is shared by both actors. The verification agent extends quiz generation, since the teacher can choose to enable it, and the correction is included in taking a quiz.

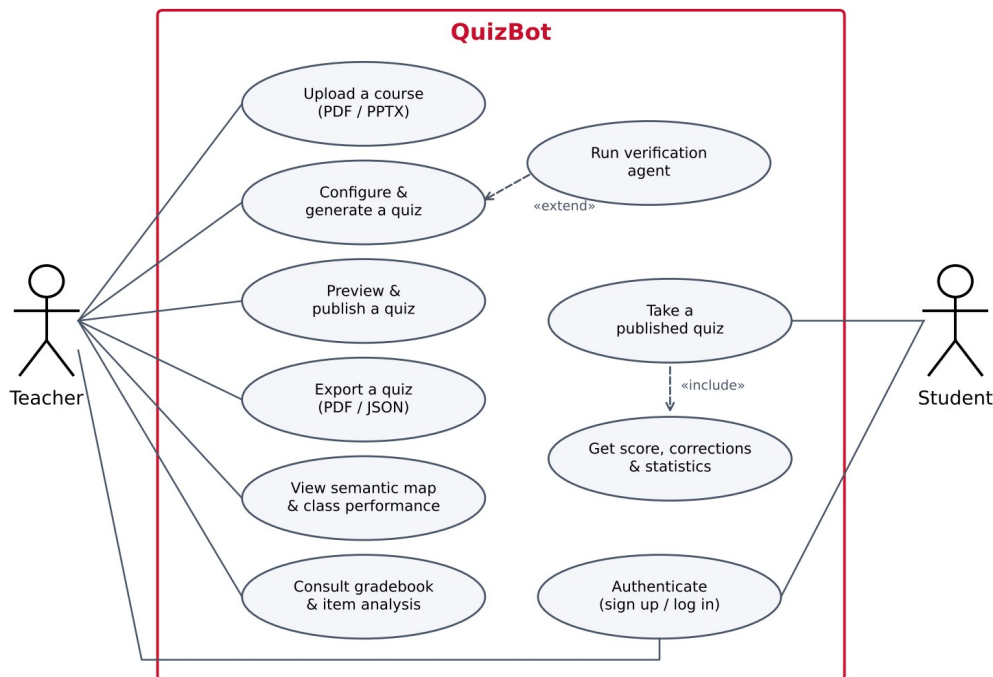


Figure 1: Global use case diagram

2.5 Textual Description of Main Use Cases

Table 3: Description of the "Generate a quiz" use case

Item	Description
Use case	Generate a quiz
Actor	Teacher
Pre-conditions	The teacher is authenticated; at least one course has been uploaded and indexed
Main scenario	1. The teacher opens the "Générer un quiz" tab. 2. He selects a document and sets the parameters. 3. He optionally enables the verification agent. 4. The system retrieves passages, calls the model, parses and checks the questions. 5. The system displays the question cards and, if the agent was used, its report (validated, regenerated, dropped).
Alternative scenarios	4a. All questions are rejected by the gates: the system shows an error with example rejection reasons and suggests regenerating. 4b. The model output is truncated: complete questions are salvaged and kept.
Post-condition	A new draft quiz is stored and visible in "Mes quiz"

Table 4: Description of the "Take a quiz" use case

Item	Description
Use case	Take a quiz
Actor	Student
Pre-conditions	The student is authenticated; at least one quiz is published
Main scenario	1. The student selects a published quiz. 2. The system displays the questions without answers or source excerpts. 3. The student answers and submits. 4. The system grades each answer and computes per-theme and per-type statistics. 5. The system displays the score and the detailed correction.
Alternative scenarios	1a. The quiz is not published: access is refused (HTTP 403). 4a. An open answer is empty: it receives a score of 0 without calling the embedding model.
Post-condition	The result is stored with its attempt number; only first attempts count in class statistics

2.6 Project Planning

The project was organised in sprints over about ten weeks. The first four sprints delivered the scope of the specification (a complete, tested application). The following sprints were dedicated to what I considered the real challenge: running the model locally and improving the quality of the questions, and then the quality of the grading. Figure 2 shows the resulting Gantt chart.

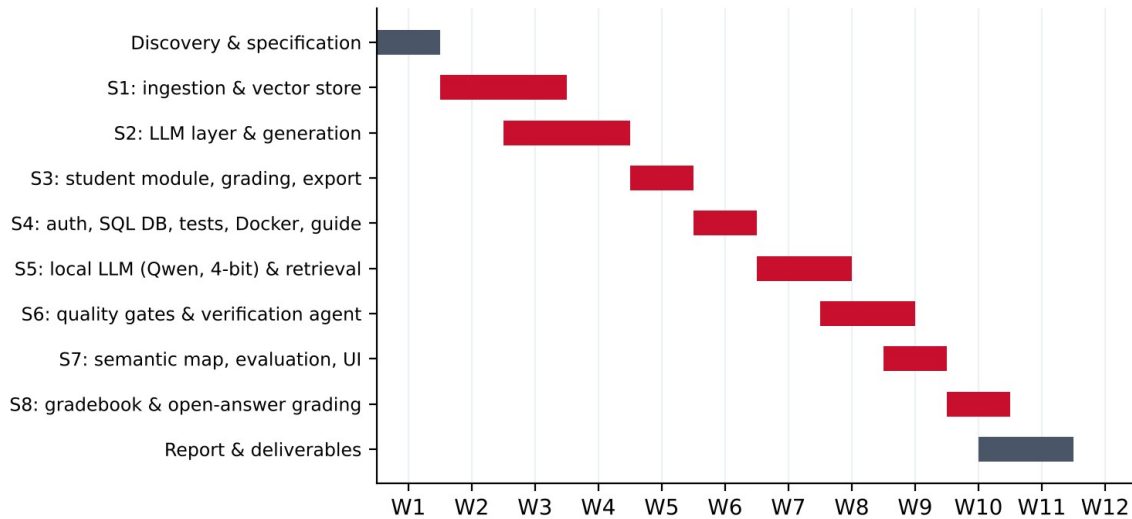


Figure 2: Project planning (Gantt chart, in weeks)

Table 5: Sprint backlog summary

Sprint	Main deliverables
Discovery	Study of RAG, of the specification and of the libraries; choice of the stack
S1	Text extraction (PDF/PPTX), chunking, embeddings, ChromaDB indexing
S2	Multi-provider LLM abstraction, mock provider, prompt and JSON parsing, first generated quizzes
S3	Student space, MCQ and open-answer grading, PDF/JSON export
S4	JWT authentication, SQL user database, integration tests, Docker, teacher guide
S5	Local Qwen2.5 provider (4-bit), document-derived query and MMR retrieval, prompt rewrite with calibration examples
S6	Programmatic quality gates, JSON salvage, verification agent with targeted regeneration
S7	Semantic map with performance overlay, evaluation script, redesign of the interface
S8	Teacher gradebook with CSV export and item analysis; three-tier grading of open answers with a grading agent and its benchmark

Conclusion

This chapter defined the actors, the functional and non-functional requirements and the planning of the project. Before moving to the design, the next chapter introduces the concepts the solution relies on.

CHAPTER 3

Theoretical Background

Introduction

QuizBot combines several techniques from natural language processing and information retrieval. This chapter presents the ones I used, with the level of detail I needed to make design decisions: language models, retrieval-augmented generation, sentence embeddings and vector search, diversity-aware retrieval, quantization, and the use of an LLM to verify another LLM's output.

3.1 Large Language Models

A large language model is a neural network, almost always based on the Transformer architecture [1], trained to predict the next token of a text. After pre-training on a very large corpus, "instruct" models are further trained to follow instructions and to answer in a conversational format with roles (system, user, assistant). This is what allows us to give the model a set of rules (the system prompt) and a task (the user prompt).

Two properties of LLMs matter directly for quiz generation. First, they **hallucinate**: a model produces fluent text, not verified facts, and it can confidently state something that is not in the source. Second, their output is **stochastic** and only loosely constrained: even when asked for "valid JSON only", a small model may add text around the JSON, forget a field or stop in the middle of a string when it reaches its token limit. Both properties shaped the robustness requirements of QuizBot.

The model I used is **Qwen2.5-Instruct** [2], an open-weight family released under the Apache 2.0 licence, available in several sizes. The 7B version is the default in QuizBot, with the 3B version as an automatic fallback when the GPU does not have enough memory.

3.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation [3] addresses the hallucination problem by giving the model the relevant source text at inference time. Instead of relying on what the model memorised during training, a retriever first selects passages from a knowledge base, and these passages are inserted in the prompt. The model is instructed to answer only from them. A typical RAG pipeline has an offline part (indexing) and an online part (retrieval and generation):

- **Indexing:** extract the text of the documents, split it into chunks, compute a vector (embedding) for each chunk and store the vectors in a vector database.
- **Retrieval:** embed a query, search the nearest chunks in the vector space and return the top-k passages.
- **Generation:** build a prompt containing the instructions and the retrieved passages, and let the LLM generate the answer.

For quiz generation, RAG has a second benefit beyond faithfulness: it lets us control **which part of the course** a question covers. The quality of the quiz depends as much on the retrieved passages as on the model.

3.3 Chunking

Documents are too long to be embedded or given to the model as a whole, so they are split into chunks. Chunk size is a trade-off: small chunks give precise retrieval but lose context; large chunks keep context but dilute the embedding. Consecutive chunks usually **overlap** so that an idea split across a boundary still appears complete in at least one chunk. The specification recommended 500 to 1000 tokens with 100 to 200 tokens of overlap; I used 800 and 150.

3.4 Sentence Embeddings

An embedding model maps a text to a dense vector so that texts with similar meaning are close in the vector space. Sentence-BERT [4] adapted BERT to produce sentence embeddings that can be compared with cosine similarity:

$$\cos(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v}) / (\|\mathbf{u}\| \|\mathbf{v}\|)$$

QuizBot uses **all-MiniLM-L6-v2**, a sentence-transformers model distilled from MiniLM [5]. It produces 384-dimensional vectors, is small enough to run on CPU, and gives good semantic similarity results for its size. The same model is used in three places: indexing chunks, retrieving passages, and grading open answers.

3.5 Vector Databases and Approximate Nearest Neighbour Search

Finding the nearest vectors exactly requires comparing the query with every stored vector. Vector databases use approximate indexes instead; **HNSW** (Hierarchical Navigable Small World) graphs [6] are among the most widely used. **ChromaDB**, the database I used, relies on HNSW and supports the cosine distance, persistent storage on disk and metadata attached to each vector (here the chunk index and the source page).

3.6 Maximal Marginal Relevance (MMR)

Plain top-k retrieval returns the k chunks closest to the query. When chunks overlap, these k chunks are often near-duplicates of each other, all coming from the same section. For a quiz, this means several questions about the same paragraph. **Maximal Marginal Relevance** [7] selects passages iteratively, trading off relevance to the query against similarity to the passages already selected:

$$\text{MMR} = \text{argmax over candidates } d \text{ of } [\lambda \cdot \text{sim}(d, q) - (1 - \lambda) \cdot \max \text{ over selected } s \text{ of } \text{sim}(d, s)]$$

With $\lambda = 1$, MMR is identical to top-k; lower values of λ favour diversity. Choosing λ is therefore a direct way to control how much of the course a quiz covers. Section 6.6 shows how I calibrated it.

3.7 Quantization of Language Models

A 7-billion-parameter model stored in 16-bit floats needs about 14 GB of memory just for its weights, far more than the 6 GB of a laptop RTX 3060. **Quantization** stores the weights with fewer bits. The `bitsandbytes` library implements the **NF4** (4-bit NormalFloat) data type introduced with QLoRA [8], which is designed for normally distributed weights; computation is done in 16-bit after de-quantization on the fly. In 4-bit, a 7B model needs roughly 4 to 5 GB of VRAM, which makes it usable on consumer hardware at the cost of a small loss in quality.

3.8 Using an LLM to Verify an LLM

A common way to improve LLM output without training is to ask a model to critique and refine its own answer (Self-Refine [9]) or to use an LLM as a judge of another model's output [10]. These approaches work, but studies of LLM judges also show their biases, such as favouring their own style or accepting plausible-looking answers. For QuizBot I combined both ideas: an LLM verifier that checks each question against the passages, preceded by **deterministic checks** that do not depend on the model at all. Section 6.3 shows that this ordering mattered in practice.

3.9 Automatic Question Generation in Education

Automatic question generation is an established research field. A systematic review by Kurdi et al. [11] notes that most systems before LLMs relied on templates and syntactic transformations, which produce questions that test recall of a sentence rather than understanding, and that the generation of good distractors is one of the hardest parts. These two points, conceptual questions and plausible distractors, became explicit rules in my prompts.

3.10 Dimensionality Reduction for Visualisation

To display a course as a map, the 384-dimensional chunk embeddings must be projected to 2D. **PCA** projects the data on the directions of largest variance and can be computed with a singular value decomposition. **t-SNE** [12] is non-linear and preserves local neighbourhoods better, which separates clusters more clearly, but it is stochastic and sensitive to its perplexity parameter. QuizBot uses t-SNE when `scikit-learn` is available and falls back to PCA otherwise.

3.11 Item Analysis (Classical Test Theory)

Once students have answered, the teacher needs to know which questions work. Classical test theory gives simple indicators for this. The **success rate** (or difficulty index) of a question is the share of students who answer it correctly. The **discrimination** of a question measures whether it separates strong and weak students; QuizBot computes it as the correlation between getting the question right and the score on the rest of the quiz (item-rest correlation). A negative value means that weaker students do better than stronger ones, which usually points to a wrong answer key or an ambiguous stem. Ebel [17] suggests that items below 0.20 discriminate poorly. Finally, **Cronbach's alpha** [18] estimates the internal consistency of the whole quiz; values above 0.7 are usually considered acceptable and above 0.8 good. These indicators are only meaningful with enough copies, which is why QuizBot computes them but raises no statistical warning below five students.

Conclusion

This chapter introduced the concepts behind QuizBot. The next chapter shows how I combined them into an architecture.

CHAPTER 4

Design of the Solution

Introduction

This chapter presents the design of QuizBot: the global architecture, the RAG pipeline, the verification agent, the data model and the main interactions, described with UML diagrams.

4.1 Global Architecture

QuizBot follows a three-layer architecture (Figure 3). The **presentation layer** is a Streamlit application with a teacher space and a student space. It never accesses data directly: everything goes through the **application layer**, a REST API built with FastAPI, which orchestrates ingestion, generation, grading, the gradebook, export and the semantic map. The **data and model layer** contains ChromaDB for the chunk vectors, a relational database (SQLite through SQLAlchemy) for app accounts, JSON files for documents, quizzes and results, the embedding model and the local LLM.

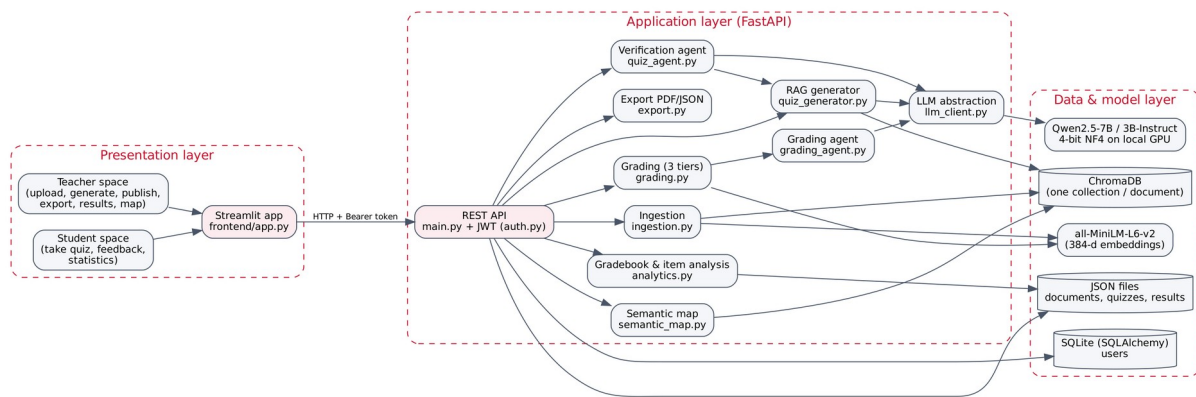


Figure 3: Global architecture of QuizBot

This separation was a deliberate choice. Because the frontend only talks HTTP, it could be replaced by a React interface without touching the backend. Because every LLM call goes through `llm_client.py`, switching from the local model to OpenAI or Mistral is a one-line change in `.env`. And because security is enforced in the API rather than in the interface, a student with a valid token cannot bypass it by calling the API directly.

4.2 Design of the RAG Pipeline

Figure 4 details the generation pipeline. The upper part runs once, when a teacher uploads a document; the lower part runs for every quiz.



Figure 4: QuizBot RAG pipeline, from upload to stored quiz

Compared with a textbook RAG pipeline, four steps are specific to this project:

- **Cleaning before chunking.** Course slides repeat headers and footers on every page (copyright banners, school name, module title). Left in place, these lines end up in every chunk and the model happily writes questions about them. The ingestion step removes them before indexing.
- **A query derived from the document.** A quiz does not start from a user question. Using the same generic query for every document ("key concepts of the course") retrieves whatever happens to resemble that sentence. QuizBot instead builds the query from the most characteristic terms of the document itself.
- **MMR selection.** The retriever fetches three times more candidates than needed, and MMR keeps a relevant but diverse subset.
- **Quality gates after generation.** Every question is checked before it is accepted.

4.3 Design of the Verification Agent

The verification agent (Figure 5) wraps the standard pipeline. After the initial batch is generated, each question goes through a loop: deterministic gates first, then an LLM verifier. If the question fails either check, the agent asks the model for a new question on the same passages, and tells it explicitly why the previous one was rejected. After two failed regenerations, the question is dropped rather than shown to a student.

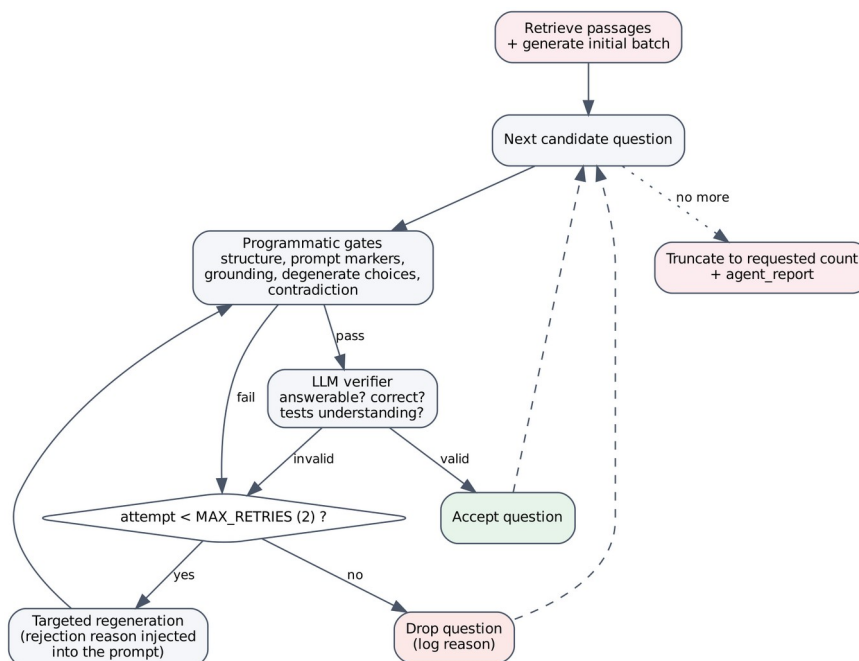


Figure 5: Generate, verify, regenerate loop of the verification agent

Two design rules guided this component. First, **deterministic checks run before the LLM verifier**: a small model can validate its own mistakes, while a rule such as "the cited excerpt must exist in the passages" cannot be fooled. Second, the verifier **fails open**: if it crashes or returns something unparseable, the question is kept and the reason is logged. An infrastructure problem in the verification step should not block a teacher from getting a quiz. The agent is optional because it costs one to three extra model calls per question.

4.4 Data Model

Figure 6 shows the main entities. Users are stored in a SQL table because they need uniqueness constraints and safe concurrent access. Documents, quizzes and results are stored as JSON files validated by Pydantic models, which was enough for the scale of an internship project and kept the data easy to inspect. Chunks and their embeddings live in ChromaDB, one collection per document.

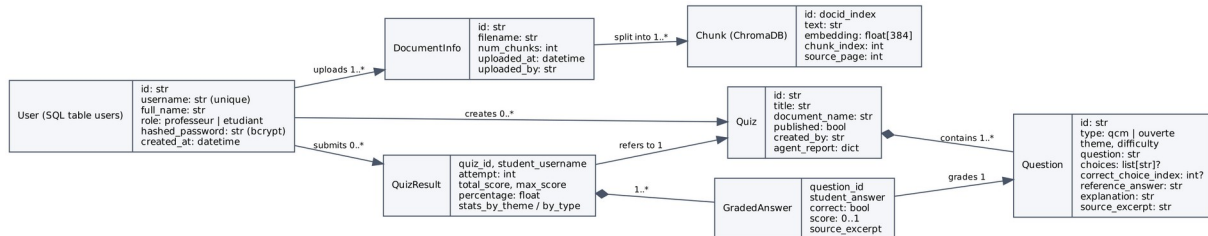


Figure 6: Class diagram of the main entities

A few attributes are worth explaining:

- `source_excerpt` (Question) holds a sentence copied verbatim from the course. It is both a proof of grounding for the teacher and the key used to place a student's answer on the semantic map.
- `agent_report` (Quiz) records how many questions were generated, accepted on the first check, regenerated and dropped. A high drop rate tells the teacher that the source document is probably too poor for the requested number of questions.
- `feedback`, `graded_by`, `similarity` and `needs_review` (GradedAnswer) record, for an open answer, the sentence shown to the student, who decided the grade (a rule, the similarity or the grading agent) and whether the teacher has to confirm it.
- `attempt` (QuizResult) numbers the submissions of a student on a quiz. Only first attempts are counted in class statistics, so that students practising several times do not inflate the results.
- `StudentQuiz` is a separate, whitelisted view of a quiz for students: it contains the questions and the choices, but not the correct index, the reference answer, the explanation or the source excerpt.

4.5 Dynamic View: Sequence Diagrams

Figure 7 shows the generation of a quiz with the agent enabled, and Figure 8 the submission and correction of a quiz by a student.

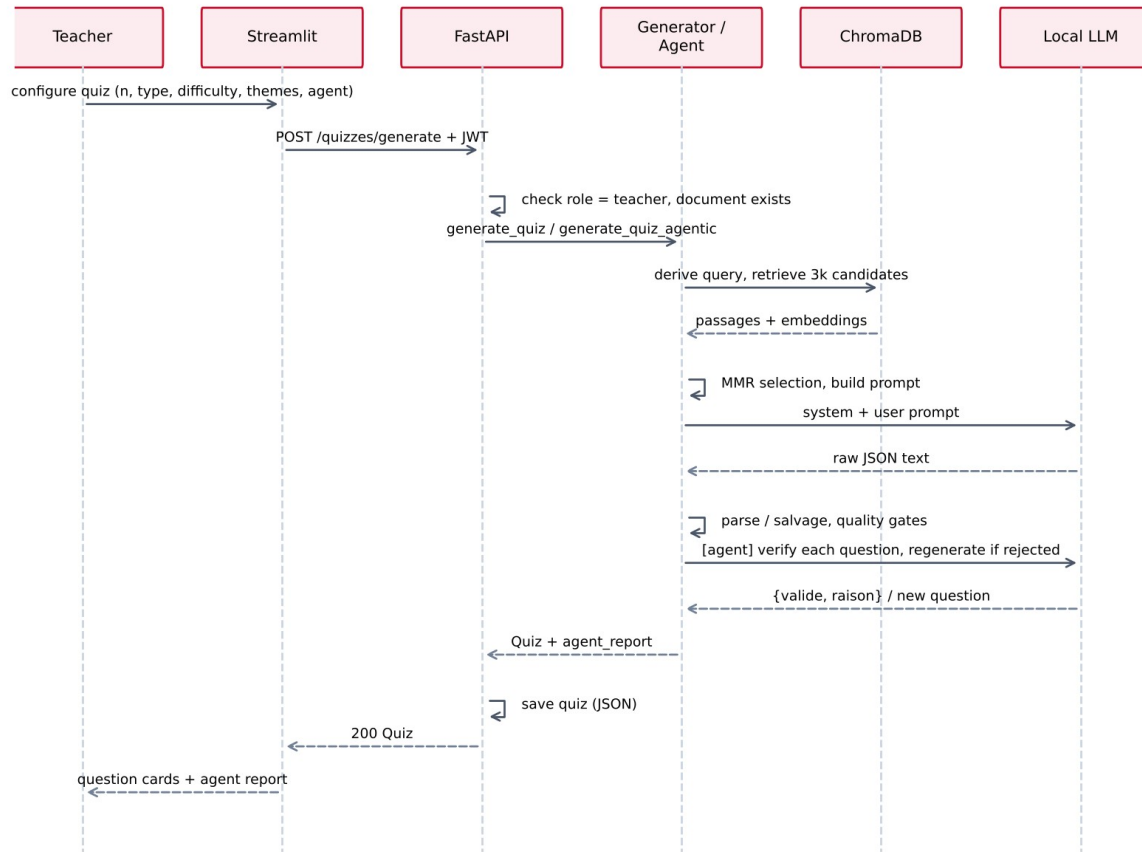


Figure 7: Sequence diagram: generating a quiz

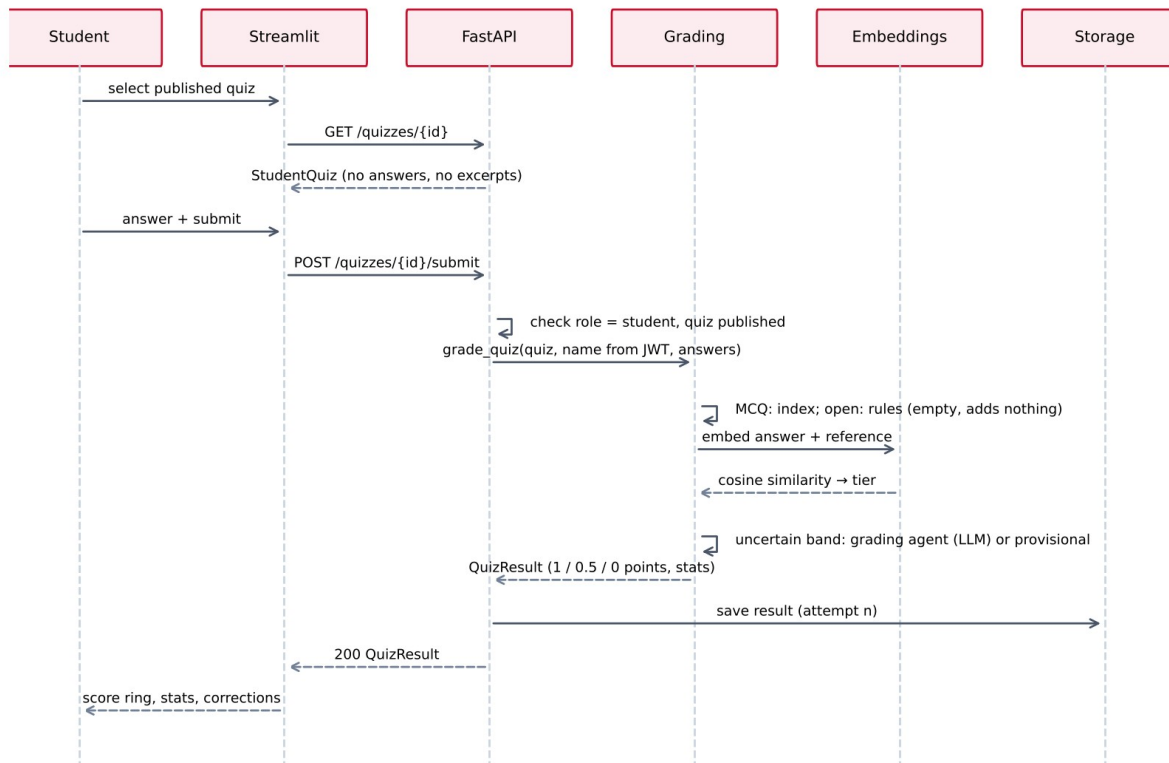


Figure 8: Sequence diagram: submitting and grading a quiz

In the submission flow, the student's name is taken from the authenticated token and never from the request body, so a student cannot submit answers under someone else's identity.

4.6 REST API Design

Table 6: Main endpoints of the QuizBot API

Method	Endpoint	Role	Purpose
POST	/auth/register	Public	Create an account (teacher code optional)
POST	/auth/login	Public	Return a JWT access token
GET	/auth/me	Any	Profile of the current user
POST	/documents/upload	Teacher	Upload, extract, chunk and index a course
GET	/documents	Teacher	List indexed documents
GET	/documents/{id}/map	Teacher	Semantic map, optionally with performance overlay
POST	/quizzes/generate	Teacher	Generate a quiz (standard or agentic)
GET	/quizzes, /quizzes/{id}	Any	List / read quizzes (students: published only, no answers)
POST	/quizzes/{id}/publish	Teacher	Publish a quiz
POST	/quizzes/{id}/submit	Student	Submit answers and get the graded result
GET	/quizzes/{id}/results	Teacher	Gradebook, class statistics and per-question analysis
GET	/quizzes/{id}/export/pdf/json	Teacher	Export with or without answers

Conclusion

This chapter described the architecture and the main design decisions. The next chapter shows how each part was implemented.

CHAPTER 5

Implementation

Introduction

This chapter presents the implementation of QuizBot. I start with the working environment and the technology choices, then go through the backend modules in the order of the pipeline, and finish with the interface and the deployment.

5.1 Working Environment

Table 7: Working environment

Element	Details
Machine	Windows laptop, NVIDIA GeForce RTX 3060 Laptop GPU with 6 GB of VRAM
Language	Python 3 with a virtual environment (venv); Python 3.11 in the Docker images
Tools	Swagger UI generated by FastAPI to test the endpoints, pytest, Docker
Model runtime	PyTorch with CUDA, Hugging Face transformers, bitsandbytes (4-bit)
Deployment	Docker and docker-compose (backend and frontend services)

5.2 Technology Choices

Table 8: Technology choices and justification

Component	Choice	Why
Backend	FastAPI + Pydantic	Automatic validation of every request and response, dependency injection for authentication, interactive documentation at /docs
Frontend	Streamlit	Fast to build a data-oriented interface in Python; enough for the two spaces, with custom HTML/CSS for the look
Extraction	PyMuPDF, python-pptx	Reliable text extraction page by page and slide by slide, including PPTX tables
Embeddings	sentence-transformers, all-MiniLM-L6-v2	Runs locally on CPU, 384 dimensions, good quality for its size, no API cost
Vector store	ChromaDB (persistent client)	Embedded database (no server), cosine HNSW index, metadata per chunk
LLM	Qwen2.5-7B-Instruct (fallback 3B), 4-bit NF4	Open weights (Apache 2.0), good instruction following and multilingual (French/English), fits in 6 GB once quantized
Users DB	SQLAlchemy + SQLite	Real relational storage with a unique constraint on usernames; PostgreSQL by changing DATABASE_URL

Component	Choice	Why
Security	bcrypt, PyJWT (HS256)	Standard password hashing and stateless tokens
Export	ReportLab	Fine control over the PDF layout in pure Python
Tests	pytest, FastAPI TestClient	Unit and end-to-end API tests with temporary data folders

5.3 Document Ingestion

The ingestion module (`ingestion.py`) turns an uploaded file into a list of chunks. It was the first module I wrote and the one I came back to most often, because every problem in the input text ended up as a problem in the questions.

5.3.1 Extraction and typography normalisation

PDF pages are read with PyMuPDF and PPTX slides with `python-pptx`, including the text of tables. Each page keeps its number so that every chunk can point back to its source page. The text is then normalised: course PDFs use typographic characters (the mathematical minus sign U+2212, en and em dashes, curly quotes, non-breaking spaces, the operator asterisk). I added this step after a Python course produced questions where $2*n-1$ was written with a Unicode minus, which is not valid Python and confused both the model and my quality checks. All these characters are replaced by their ASCII equivalents.

5.3.2 Removing repeated headers and footers

The first quizzes I generated contained questions about the copyright line printed on every slide. The function `strip_repeated_boilerplate` removes such lines with two mechanisms:

1. Any line starting with "©" or "(c)" is removed unconditionally. Some documents mix several banner variants across chapters, each below the global repetition threshold, so a frequency rule alone was not enough.
2. Any other line of at least 8 characters that appears on at least 15% of the pages (and at least 2 pages) is considered boilerplate and removed. This rule is only applied to documents of 3 pages or more, so that short documents do not lose real content.

A safety net keeps the original text if the cleaning would remove everything.

5.3.3 Chunking with structure preservation

The cleaned pages are concatenated into a list of (word, page) pairs and cut into windows of 800 words with a step of 650 (150 words of overlap). I approximate one token by one word, which is enough for chunking and avoids loading a tokenizer. Two details made a visible difference in the questions:

- Line breaks are preserved through a special marker during the word-level split. Without it, the end of a bullet point and the title of the next slide were merged into one incoherent "sentence".
- Lines broken only because of the slide width are merged back (`_merge_wrapped_lines`): a line that does not start with a bullet or a number and follows a line without final punctuation is treated as a continuation.

Each chunk stores its dominant source page. Table 10 in Chapter 6 gives the effect of this pipeline on the four courses I used.

5.4 Indexing and Retrieval

`vectorstore.py` is a thin layer over ChromaDB with one collection per document (`doc_<id>`), using the cosine distance. Chunk embeddings are computed by `embeddings.py`, which loads the sentence-transformers model once (singleton) and normalises the vectors. The query method returns, in addition to the text and the distance, the embedding of each hit, because MMR needs to compare candidates with each other.

5.4.1 Document-derived query

When the teacher gives themes, the query is simply built from them. Otherwise, `_derive_document_query` extracts the most characteristic terms of the document. For each term of at least four letters that is not a French or English stop word, it computes its frequency and the number of distinct chunks in which it appears, and scores it with:

$$\text{score}(\text{term}) = \text{frequency}(\text{term}) \times \text{number of chunks containing term}$$

Terms that appear in only one chunk are ignored: a term that is frequent but concentrated in one place is probably anecdotal, while a term spread across the course characterises it. The ten best terms form the query, for example "Concepts, definitions and key notions about: list, function, string, ..." for the Python course.

5.4.2 MMR selection

The retriever asks ChromaDB for three times more candidates than needed ($k = \max(\text{number of questions}, 4)$), so at least 12 candidates), then `_mmr_select` keeps k of them with $\lambda = 0.55$. The first passage is the most relevant one; each next passage maximises $\lambda \cdot \text{relevance} - (1 - \lambda) \cdot \text{max similarity to the passages already chosen}$. If embeddings are missing (an older collection), the function falls back to the original order.

```
selected = [max(hits, key=lambda h: h["_relevance"])]
remaining = [h for h in hits if h is not selected[0]]
while remaining and len(selected) < top_k:
    best, best_score = None, float("-inf")
    for cand in remaining:
        redundancy = max(cosine_similarity(cand["embedding"], s["embedding"])
                        for s in selected)
        score = lambda_mult * cand["_relevance"] - (1 - lambda_mult) * redundancy
        if score > best_score:
            best, best_score = cand, score
    selected.append(best); remaining.remove(best)
```

5.5 The LLM Abstraction Layer

All model calls go through a single function, `generate_completion(system_prompt, user_prompt)`. The provider is chosen from the `LLM_PROVIDER` variable among five implementations of a common abstract class: `openai`, `mistral`, `huggingface`, `local` and `mock`. The rest of the code never knows which model answers.

5.5.1 Local provider

The `LocalLLMProvider` loads Qwen2.5-7B-Instruct with transformers and a bitsandbytes configuration (4-bit, NF4, float16 compute) and places it on the GPU with `device_map="auto"`. The model is loaded once per server process and shared between requests. Loading a 7B model on a 6 GB laptop GPU is at the limit, so the provider tries the configured model first and, on an out-of-memory or loading error, frees the GPU cache and loads the fallback model (Qwen2.5-3B-Instruct) instead of leaving the application unusable. A small script, `test_local_llm.py`, loads the model and generates one sentence; I used it to debug the CUDA and bitsandbytes installation separately from the rest of the application.

The prompt is formatted with the model's own chat template, generation uses sampling with a temperature of 0.4, and only the newly generated tokens are decoded.

5.5.2 Token budget

With a fixed limit on new tokens, a 3-question quiz worked but a 10-question quiz was cut in the middle of a string, and the whole JSON was lost. The provider now reads the requested number of questions from the prompt and uses $\max(3600, n \times 420)$ tokens, 420 being a measured upper bound for one complete MCQ (stem, four choices, answer, explanation and excerpt) plus a margin.

5.5.3 Mock provider

The mock provider produces questions without any model, by reusing real sentences of the retrieved passages: the correct choice comes from the passage and the distractors from other passages. It does not understand anything and its questions are poor by design, but it lets the whole pipeline (and the 108 tests) run on any machine, without a GPU or an API key. It also implements a fake verifier that checks whether the answer text appears in the passages, so that the agent loop can be tested end to end.

5.6 Prompt Engineering

My first prompt asked the model to "generate questions from the passages". The results were exactly what the literature warns about: questions whose answer was a sentence of the course copied word for word, and distractors taken from unrelated parts of the document. I rewrote the system prompt around explicit quality rules:

- each question must test the understanding of a concept, a definition, a cause and effect relation or an application, and must still make sense to a student who does not have the passage in front of him;
- the model must never copy a sentence of the passage in the stem, the choices or the answer, except in the `source_excerpt` field, which must be a verbatim sentence of at least six words and never the label of a passage;
- distractors must be plausible confusions close to the concept (a neighbouring definition, a common student mistake), never sentences about another topic;
- the answer is a strict JSON object with a fixed schema.

Rules alone were not enough for a small model, so the user prompt also contains a **calibration example** on a fictitious passage about min-max normalisation: a bad question (copied answer,

off-topic distractors) and a good one (a "why" question with three credible distractors), each with an explanation of why it is good or bad. The prompt also states the language to use. Course material in engineering is often bilingual, and the small model drifted to English even when prompted in French, so `_detect_language` counts frequent French and English function words in the passages and the prompt ends with an explicit "LANGUE = français" or "anglais".

One last detail came from debugging: the passages must be the **last** block of the prompt. When I appended instructions after them, the model read those instructions as part of the last passage.

5.7 Parsing and Salvaging the Model Output

The parser (`_parse_llm_response`) removes markdown code fences, extracts the first complete top-level JSON object by counting braces (small models often add text before or after it), and validates each question with Pydantic. A malformed question is skipped instead of failing the whole quiz. If the JSON is truncated, `_salvage_question_objects` walks through the "questions" array character by character, tracking strings and escape characters, and recovers every object that was fully closed. In practice this turned many "failed" generations into quizzes with slightly fewer questions.

5.8 Programmatic Quality Gates

The function `_quality_reject_reason` checks a question against the passages it was generated from and returns a human-readable reason, or nothing if the question is valid. The checks are ordered from the cheapest to the most expensive:

Table 9: Programmatic quality gates

Gate	Rule	Problem it targets
Structure	MCQ has at least 3 non-empty choices and a valid answer index	Broken JSON fields
Prompt artifacts	No field contains "[PASSAGE n]" or "NUM_QUESTIONS"; tolerant to typos such as "[PASSEAGE 1]" or "[PASSEAU 2]"	The model copying the prompt's labels instead of course text
Grounding	An excerpt of 4+ words must share a window of 6 consecutive words with the passages (after normalisation)	Invented or paraphrased "proof"
Degenerate choices	No duplicate choices, no permutations of the same words, average lexical overlap between choices ≤ 0.6 ; code choices compared raw	Four versions of the same answer
Contradiction	If the correct choice uses the <code>**</code> operator, the stem or explanation must use it too	Answer that does not match its own question (e.g. $2^{**}n - 1$ for $2*n - 1$)

The tolerance to typos in the artifact gate was not designed in advance: it comes directly from questions generated by the local model, where the labels "[PASSEAGE 1]" and "[PASSEAU 2]" appeared in the source excerpts. In the standard pipeline, questions that fail a gate are

logged and dropped; if all of them fail, the API returns an error with up to three example reasons, which is much more useful to the teacher than an empty quiz.

5.9 Verification Agent

`quiz_agent.py` implements the loop of Figure 5. Its verifier prompt asks the model to check three points: the question can be answered from the passages, the indicated answer is correct, and the question tests understanding rather than the memorisation of a copied sentence. The answer must be a JSON object `{"valide": true|false, "raison": "..."}.`

When a question is rejected, the regeneration prompt reuses the standard prompt for a single question of the same type, difficulty and theme, and adds a note with the rejection reason and the rejected question, asking for a different one. Any text coming from the model that is re-injected into a prompt goes through `_neutralize_passage_labels`, otherwise a label copied by the model would look like a real passage to the verifier. For the same reason, in the verification prompt the passages come after the question: placed after them, the indicated answer would read as course content and the verifier would validate it against itself.

```
for question in candidate_questions:
    current, accepted = question, False
    for attempt in range(MAX_RETRIES + 1):          # MAX_RETRIES = 2
        result = verify_question(current, passages) # gates, then LLM verifier
        if result.valide:
            accepted = True; break
        if attempt < MAX_RETRIES:
            replacement = _regenerate_question(config, passages, current,
result.raison)
            if replacement is None: break
            current = replacement; report["regenerated"] += 1
    (final_questions if accepted else dropped).append(current)
```

5.10 Automatic Grading

MCQs are graded by comparing the selected index with the correct one (or the text, case-insensitively). Grading open answers was harder, and I rewrote it at the end of the internship.

5.10.1 From a similarity score to three tiers

In the first version, an open answer scored its cosine similarity with the reference answer, and was counted as correct above 0.62. Looking at real copies showed two problems: an answer like "je ne sais pas" still earned 0.1 to 0.4 point, and a student who simply pasted the question back was often judged correct, because the stem shares many words with the reference. I replaced this with the scale a human grader uses: **right (1 point)**, **partial (0.5)** or **wrong (0)**, decided in this order:

1. an empty answer is wrong;
2. an answer that adds no meaningful word to the question (after removing stop words and accents) is wrong, without computing anything;
3. otherwise the similarity decides the clear cases: below 0.45 the answer is wrong, from 0.80 it is right;

4. between the two thresholds, the similarity cannot decide: the **grading agent** is asked; if it cannot answer, the copy receives a provisional 0.5 point marked "to be confirmed by the teacher".

A question saved without a reference answer (a generation defect: 6 of the 18 open questions stored during the internship) is no longer compared with an empty string: the agent grades it from the course passage if one exists, otherwise the answer is flagged for the teacher.

5.10.2 Grading agent

`grading_agent.py` sends the configured LLM the question, the student's copy, the reference answer and the course passage, with a rubric for the three verdicts, and expects `{"verdict": "juste|partiel|faux", "justification": "..."}.` The justification is shown to the student. Because a copy is text written by a student, it is treated as untrusted data:

- the copy is placed between markers that are removed from the copy itself, so a student cannot close the block, and passage labels are neutralised;
- the copy comes **before** the reference and the passage, for the same reason as in the quiz verifier: placed after them, it would read as course content;
- a copy that addresses the grader ("note pour le correcteur", "ignore les consignes", "tous les points", "20/20"...) is never shown to the agent: it is graded on similarity alone, which text cannot manipulate, and flagged for the teacher;
- an unusable answer or any error (including a CUDA out-of-memory error) keeps the provisional grade, so a submission never fails because of the agent;
- at most 6 agent calls per copy, a 160-token budget and a lock around the local model, so two students submitting at the same time never run the model concurrently on the GPU.

Each graded open answer records who decided it (rule, similarity with its value, or agent), which the teacher can see in the detailed copy. Scores are then aggregated by theme and by question type to show the student his weak areas. Section 6.5 measures the effect of this new grading.

5.11 Teacher Gradebook and Item Analysis

Until late in the project, results were saved but nothing read them back except the semantic map: the teacher had no way to see the grades. I added a "Résultats" tab backed by a new endpoint, `GET /quizzes/{id}/results`, implemented in `analytics.py`.

- **Gradebook.** One row per submission with its attempt number; class statistics (number of students, mean, median, students above 50%, retries) are computed on first attempts only. A "To check" column counts the provisional grades of each copy. The teacher can open a detailed copy, where an MCQ answer is shown as the choice the student saw.
- **CSV export** for a French spreadsheet (";" separator, decimal comma, UTF-8 BOM). Names starting with `=`, `+`, `-` or `@` are prefixed so that they cannot be executed as formulas when the file is opened (CSV injection).
- **Item analysis.** For each question: success rate, mean points, blanks, distribution of the MCQ choices, and discrimination (item-rest correlation). Questions are flagged with a short French explanation: too easy ($\geq 90\%$), too hard ($\leq 25\%$), weak or negative discrimination (< 0.20), a distractor chosen more often than the key, a distractor never

chosen, often left blank ($\geq 30\%$). The reliability of the whole quiz is given by Cronbach's alpha. Below 5 copies the figures are still shown but no statistical flag is raised, because they would mostly be noise. A question without a reference answer is always flagged.

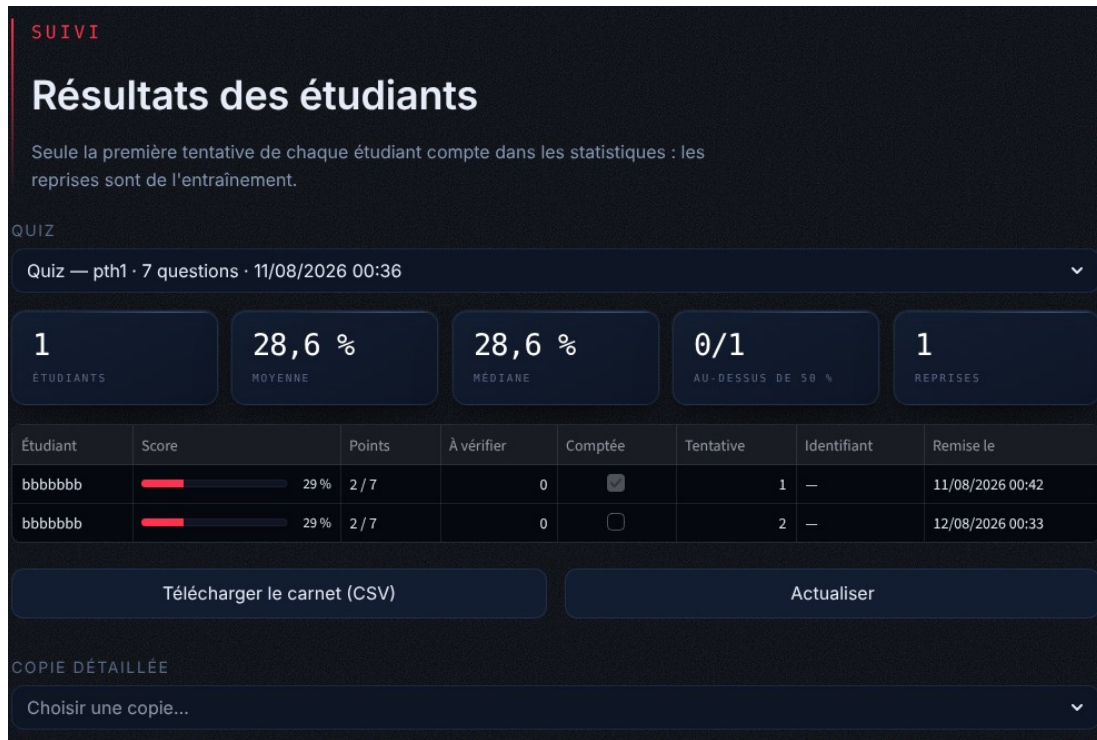


Figure 9: Teacher space: gradebook of a quiz (real results of the Python quiz)

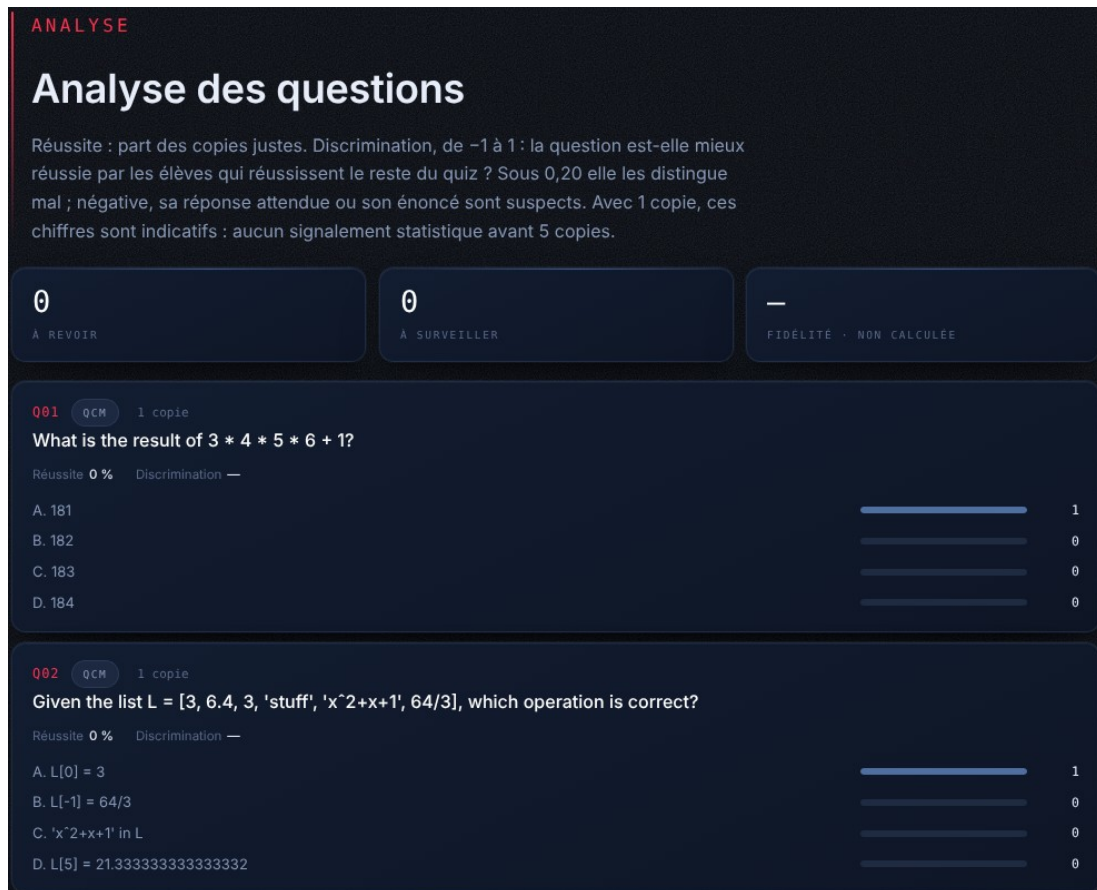


Figure 10: Teacher space: per-question analysis (indicative here, with a single copy)

5.12 Semantic Map of a Course

The semantic map was not in the specification; I added it because I wanted the teacher to see what a quiz actually covers. `semantic_map.py` takes the embeddings of all the chunks of a document and projects them to 2D with t-SNE (perplexity adapted to the number of chunks) or, without scikit-learn, with a PCA computed by SVD in NumPy. Coordinates are normalised to the unit square and displayed on an interactive canvas where hovering a point shows the passage.

With the performance overlay, each graded answer is attached to the chunk that contains its source excerpt, found by matching 6-word n-grams against the full text of every chunk (the same guarantee as the grounding gate). Each point is then coloured by the average score of the students on the questions coming from that passage, using first attempts only. A red area shows a part of the course that students did not understand; a grey area was never evaluated, which is a different piece of information.

5.13 Export

`export.py` produces a JSON file (optionally without answers, for integration in an LMS) and a printable PDF with ReportLab, with the correct answers in bold and the explanations when requested. All course content is escaped before being passed to ReportLab, whose paragraphs interpret an XML-like markup: an HTML course containing tags such as `<TITLE>` otherwise broke the PDF generation.

5.14 Authentication and Security

- Passwords are hashed with bcrypt; tokens are JWT signed with HS256 and expire after 8 hours. The secret key is read from `.env`.
- Registering as a teacher can require a code (`PROFESSOR_SIGNUP_CODE`), so that anyone cannot give himself the teacher role.
- Each endpoint declares the roles it accepts through a FastAPI dependency (`require_role`).
- Students only receive the whitelisted `StudentQuiz` view, never see unpublished quizzes, and the identity on a result always comes from the token.
- Configuration values are read with a helper that treats an empty variable as missing, after an empty `DATABASE_URL=` line in `.env` silently broke the start-up.

5.15 User Interface

The interface is a single Streamlit application that shows a different space depending on the role in the token. I redesigned it during the last sprint with a custom dark theme in the ESPRIT colours. The following screenshots were taken on the application with the quizzes actually generated during the internship (the gradebook screens are shown in Section 5.11).

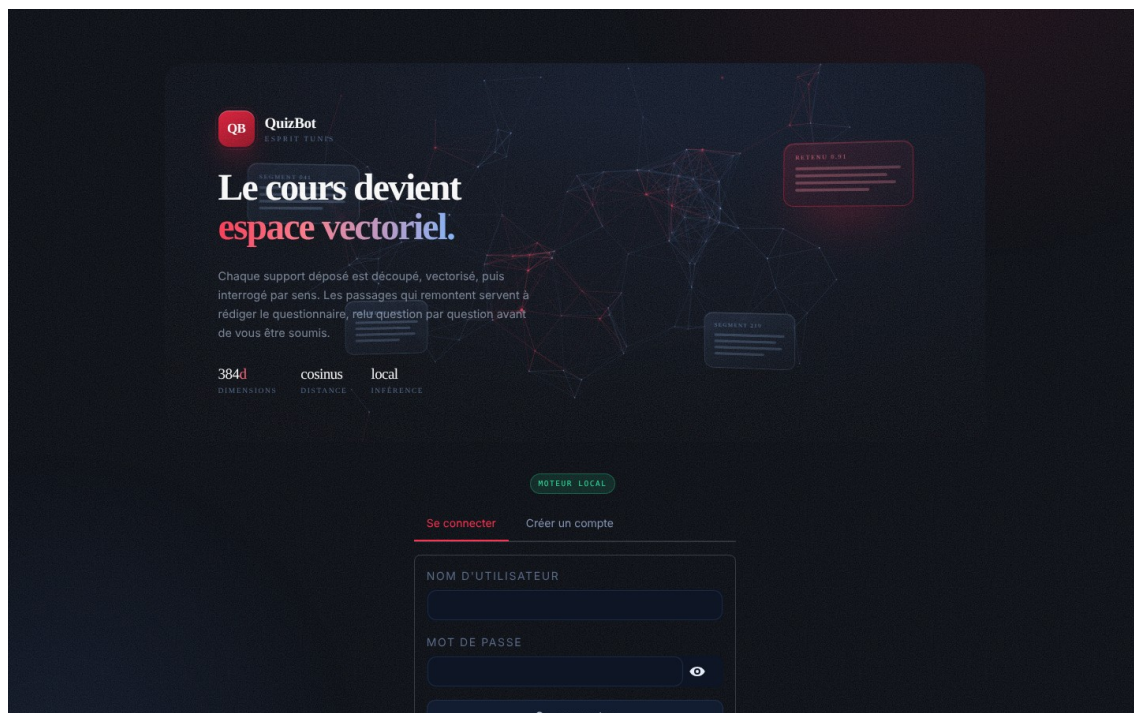


Figure 11: Home and login screen

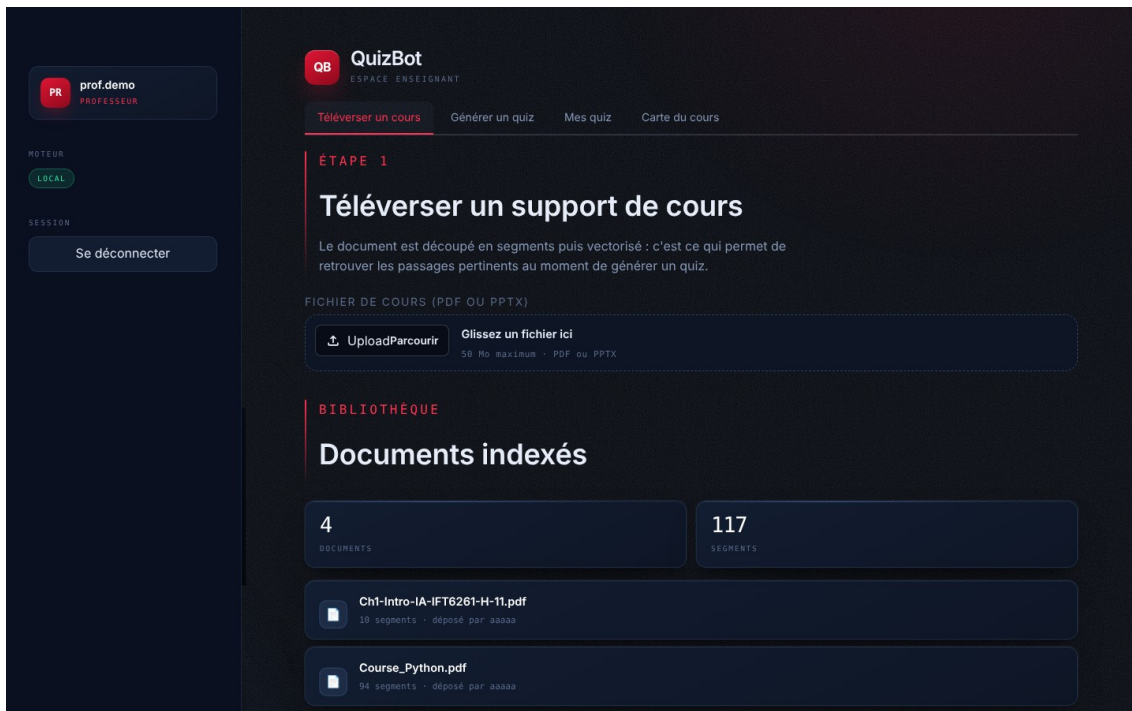


Figure 12: Teacher space: uploading a course and list of indexed documents

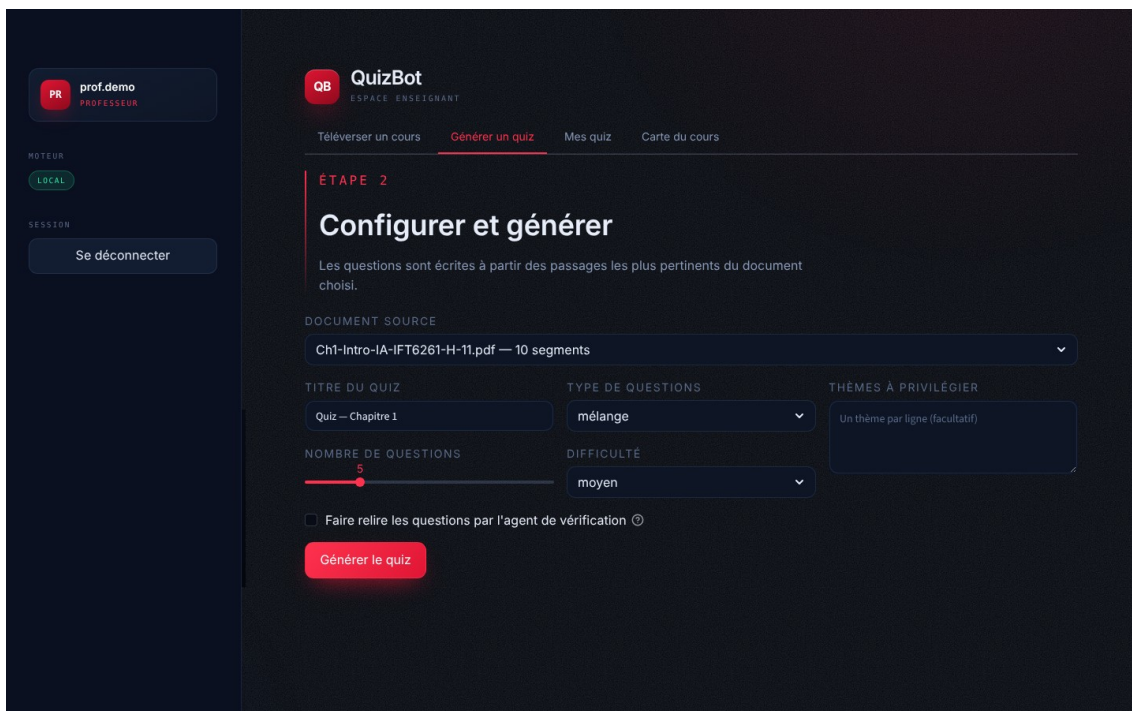


Figure 13: Teacher space: quiz configuration, with the option to enable the verification agent

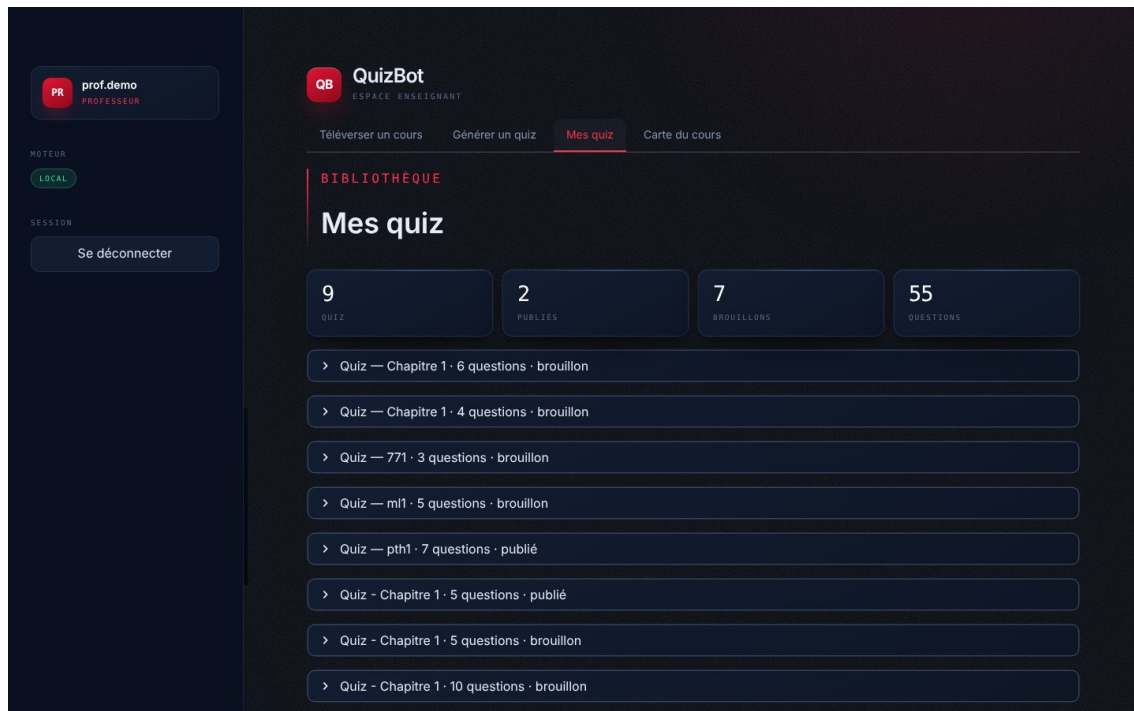


Figure 14: Teacher space: library of generated quizzes (publish, export PDF/JSON)

QUESTION 01

La base de règles dans un système expert contient principalement des règles de production et des connaissances heuristiques. Quel est le but principal de ces règles?

- ☒ DÉTERMINER LA SOLUTION D'UN PROBLÈME
- ☐ ANALYSER DES DONNÉES DE MANIÈRE STATISTIQUE
- ☐ FAIRE DES PRÉDICTIONS BASÉES SUR DES MODÈLES PRÉEXISTANTS
- ☐ GÉRER DES BASES DE CONNAISSANCES

QUESTION 02

Quel est le but principal de la reconnaissance des formes dans l'IA ?

- ☐ ANALYSER DES DONNÉES DE MANIÈRE STATISTIQUE
- ☒ FAIRE DES PRÉDICTIONS BASÉES SUR DES MODÈLES PRÉEXISTANTS
- ☐ GÉRER DES BASES DE CONNAISSANCES
- ☐ PERMETTRE L'INTERACTION ENTRE L'HOMME ET LA MACHINE

QUESTION 03

Quel est le but principal de l'apprentissage automatique dans l'IA ?

- ☒ ANALYSER DES DONNÉES DE MANIÈRE STATISTIQUE
- ☐ FAIRE DES PRÉDICTIONS BASÉES SUR DES MODÈLES PRÉEXISTANTS
- ☐ GÉRER DES BASES DE CONNAISSANCES
- ☐ EFFECTUER DES TÂCHES QUI NE POUVAIENT PAS L'ÊTRE AUPARAVANT

QUESTION 04

Comment les systèmes d'apprentissage automatique peuvent-ils aider à la reconnaissance des formes dans l'IA ?

Rédigez votre réponse...

QUESTION 05

Quels sont les principaux objectifs de la robotique dans l'IA ?

Rédigez votre réponse...

Remettre ma copie

Figure 15: Student space: taking a published quiz (MCQs and open questions)

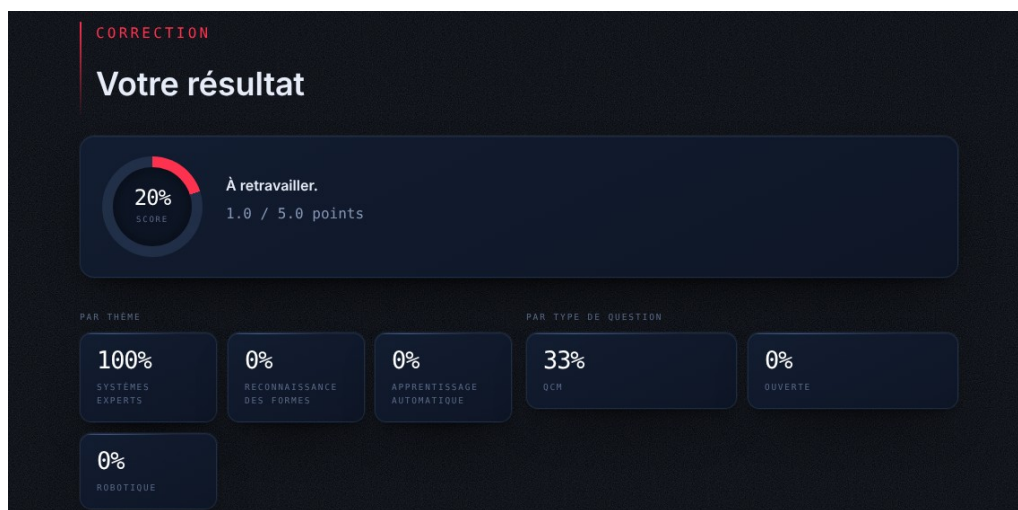


Figure 16: Student space: score and statistics by theme and question type

Quiz — Chapitre 1

Document source : Ch1-Intro-IA-IFT6261-H-11.pdf

Généré le : 02/10/2026 22:22

Question 1 (QCM — moyen)

Les systèmes experts sont utilisés pour...

- **A. Définir des règles de production et des connaissances heuristiques, sélectionner des règles et faits pour générer de nouvelles règles et faits. (bonne réponse)**
- B. Analyser des données statistiques et identifier des tendances
- C. Créer des programmes qui génèrent leurs propres connaissances en se modifiant à partir de leurs succès et erreurs.
- D. Effectuer des tâches de manière plus rapide et plus précise que les systèmes d'apprentissage analytique

Explication : Les systèmes experts utilisent des règles de production pour résoudre des problèmes. Elles sélectionnent des règles et faits pour générer de nouvelles règles et faits.

Question 2 (QCM — moyen)

Le raisonnement procédural implique...

- **A. L'utilisation de règles de production pour résoudre des problèmes. (bonne réponse)**
- B. L'application d'algorithmes rigides et exhaustifs
- C. La génération de nouvelles connaissances en se modifiant à partir de ses succès et erreurs
- D. La réutilisation de cas similaires pour résoudre de nouveaux problèmes

Explication : Le raisonnement procédural implique l'utilisation de règles de production pour résoudre des problèmes. Il génère de nouvelles connaissances en sélectionnant des règles et faits.

Question 3 (OUVERTE — facile)

Quels facteurs influencent l'acquisition de connaissances d'un expert ?

Réponse attendue :

Question 4 (QCM — moyen)

Les systèmes d'apprentissage analytique permettent de...

- A. Effectuer des tâches qui ne pouvaient pas l'être auparavant.
- B. Effectuer de façon plus efficace des tâches déjà effectuées.
- **C. Effectuer des tâches de manière plus rapide et plus précise. (bonne réponse)**
- D. Effectuer des tâches de manière plus rapide et plus précise que les systèmes d'apprentissage synthétiques

Explication : Les systèmes d'apprentissage analytique permettent de rendre les tâches plus rapides et plus précises.

Question 5 (OUVERTE — facile)

Quel est le but des systèmes d'apprentissage synthétiques ?

Figure 17: First page of a quiz exported as PDF (generated by Qwen2.5 on the AI introduction course)

5.16 Deployment

Besides the manual installation, the project contains two Dockerfiles and a docker-compose file that start the backend (port 8000) and the frontend (port 8501) with one command. The data folder is mounted as a volume so that documents, the Chroma index, quizzes and results persist. The frontend reaches the backend through the service name (API_BASE_URL=http://backend:8000). A short user guide for teachers (GUIDE_UTILISATION.md) explains the full workflow in French.

Conclusion

This chapter presented the implementation of every module of QuizBot. The next chapter evaluates the result.

CHAPTER 6

Tests, Evaluation and Results

Introduction

A quiz generator is hard to evaluate: "it runs" says nothing about whether the questions are good. In this chapter I present three kinds of evidence: the automated test suite, which checks that the system behaves as specified; measurements on the data produced during the internship; and a qualitative analysis of the questions, including the failures.

6.1 Automated Tests

The test suite contains **150 tests** written with pytest, spread across 17 files (Figure 18). They run in about 30 seconds and all pass. Heavy dependencies (ChromaDB, sentence-transformers) are replaced by fakes, the LLM is the mock provider, and every test uses a temporary data folder and a temporary database, never the development data.

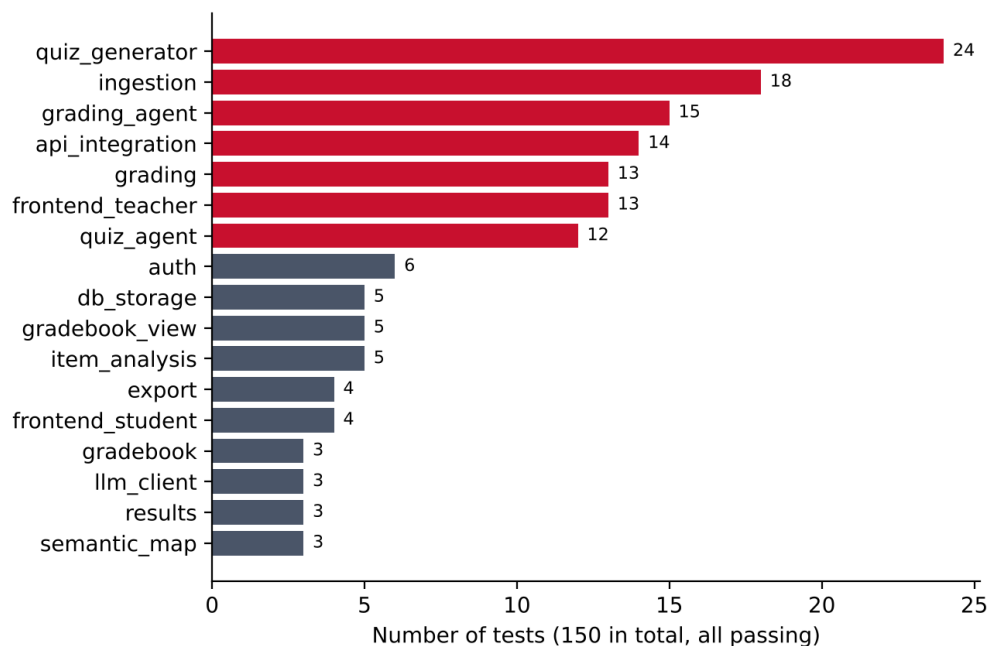


Figure 18: Distribution of the 150 automated tests by module

- **Unit tests** cover extraction, boilerplate removal, chunking and line merging (18 tests on ingestion, including a real PDF fixture), the parser and the salvage of truncated JSON, every quality gate, MMR selection and the document-derived query (24 tests on the generator), the agent loop (12 tests: acceptance of grounded questions, rejection of fabricated or ungrounded answers, the end-to-end loop, error messages that name the drop reasons, prompt ordering and label neutralisation), grading (13) and the grading agent (15, including copies that try to address the grader or close the data block), the gradebook and item analysis (13), export, authentication and the database.
- **Integration tests** (14) run the complete scenario through the API: register, log in, upload, generate, publish, submit, grade, export, and they check access control: every endpoint

requires a token, a student cannot call teacher endpoints nor retrieve the answers of a quiz with his token, the teacher sign-up code is enforced, and only first attempts are counted in the map overlay.

- **Interface tests** (17) run the Streamlit application with AppTest, without a browser, to check the teacher and student screens.

Most tests were written while fixing a real bug, so they also serve as documentation of the problems met. For example, the test on HTML tags in the PDF export, the tests on the Unicode minus sign and the tests on the "[PASSEAU 2]" artifacts all come from actual failures.

6.2 Ingestion on Real Courses

Table 10: Effect of the ingestion pipeline on the four test courses

Course	Pages	Lines removed	Words kept	Chunks
Introduction to AI (Ch1)	46	8	4,933	10
Python course	254	140	51,513	94
Machine learning methods	57	58	1,983	4
HTML course	22	7	4,062	8

The numbers show how different course material can be. The machine learning slides lose 58 lines (8% of the text) to repeated headers, while the AI course loses only 8. They also show a limit: the 57-page machine learning course contains fewer than 2,000 words of text, because most of its content is in images and formulas. It produces only 4 chunks, which explains why quizzes on this document were the hardest to get right (Section 6.3).

6.3 Quality Gates: Retrospective Replay

Throughout the internship, every generated quiz was stored. At the end, I re-ran the current quality gates on all of them, using the chunks of their source document as passages. This gives a simple but honest measurement of what the gates catch. Out of **55 stored questions**, **15 would be rejected today** (Figure 19).

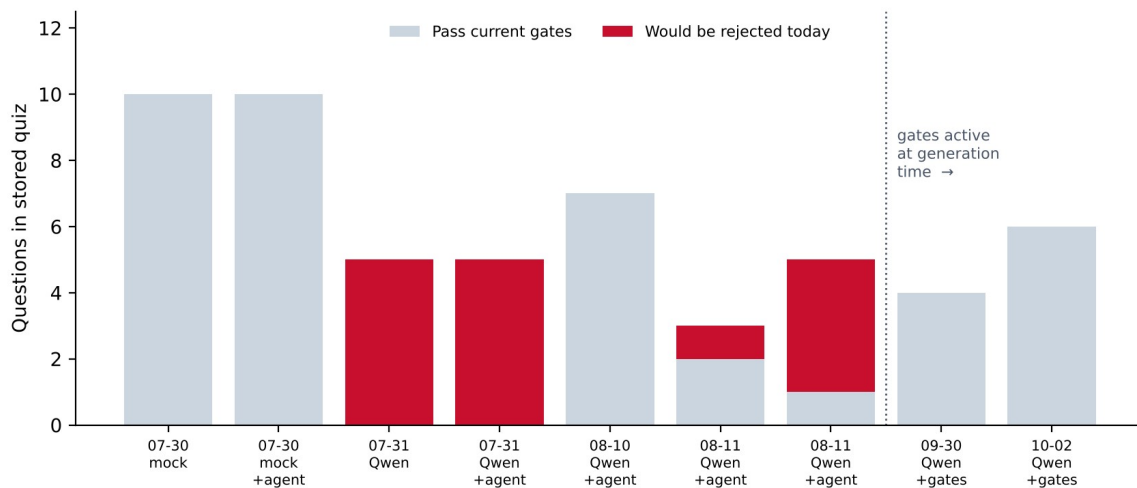


Figure 19: Stored quizzes, in chronological order, re-checked by the current quality gates

Several observations come out of this replay:

- **Prompt artifacts were the main defect of the early local-model quizzes.** 14 of the 15 rejections are source excerpts that contain a passage label ("[PASSAGE 1]", "[PASSEAGE 1]", "[PASSEAU 2]") instead of a sentence of the course. Both quizzes of 31 July are entirely affected.
- **The LLM verifier alone did not catch them.** The quiz of 31 July generated with the agent was validated 5/5 on the first check, yet all five excerpts are labels. This is the observation that convinced me to run deterministic gates before the verifier.
- **The contradiction gate caught a real wrong answer.** In a quiz on the Python course, the question asked for the list comprehension generating $2*n-1$, and the answer marked correct was `[2**n - 1 for n in range(1, 11)]`. The gate rejects it because the answer uses `**` while the question does not.
- **Quizzes generated after the gates were active pass them,** which is expected since rejected questions were already removed at generation time. In the run of 2 October, 10 questions were generated and 4 were dropped by the gates, leaving 6.
- **The mock quizzes pass all the gates.** This is a limit worth stating: the gates check structure and grounding, not pedagogical quality. A question whose answer is a sentence copied from the course is perfectly grounded. Detecting that is the job of the verifier's third criterion.

6.4 Verification Agent

Over the four quizzes generated with the agent and the local model, the agent processed 21 candidate questions. 14 were accepted on the first check (67%), 6 were accepted after one or two regenerations, and only 1 was dropped, for 9 regeneration calls in total (Figure 20).

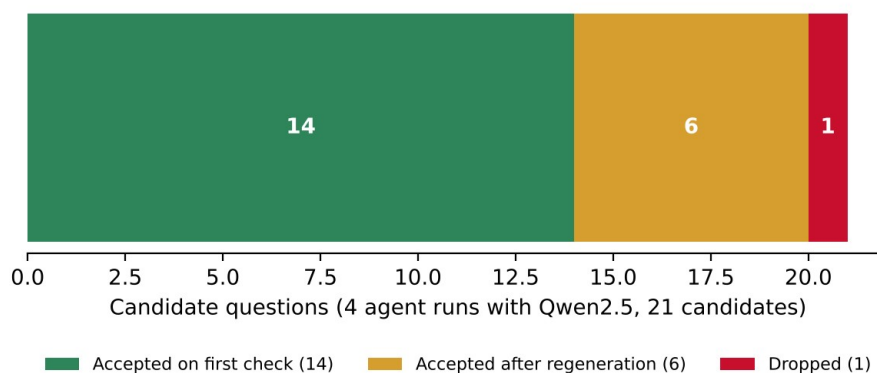


Figure 20: Outcome of the verification agent on 21 candidate questions

The agent therefore recovers most of the questions it rejects instead of simply removing them, at the cost of extra model calls. However, these numbers must be read together with Section 6.3: some of the questions it accepted were later rejected by the deterministic gates. The agent improves quality, but a small model judging its own output is not a sufficient control on its own.

6.5 Grading of Open Answers

To measure the new grading (Section 5.10), I built a small benchmark (`scripts/grading_benchmark.json`): 48 answers to 7 open questions, the questions being written by the local model or taken from the stored quizzes. The answers were written by hand to cover the typical cases (correct, reformulated, partial, vague, off-topic, "I don't know", pasted question) and are labelled by construction; they are not real student copies. `scripts/evaluate_grading.py` compares the grade given by each method with the label (right 1, partial 0.5, wrong 0). The agent was the real Qwen2.5-3B-Instruct in 4-bit on the laptop GPU.

Table 11: Open-answer grading on 48 labelled answers

Measure	Old rule (similarity)	Three tiers	Tiers + agent	Agent alone
Mean error (points)	0.320	0.167	0.115	-
Exact verdicts (of 48)	38	33	38	24
Wrong answers counted right	2	0	0	5
Right answers counted wrong	2	1	1	0
Provisional grades left to the teacher	0	19	2	-

The old rule gave on average 0.26 point to non-answers and 0.51 point to a pasted question; both drop to 0 with the three tiers. The tiers alone are cautious: they never count a wrong answer as right, but they leave 19 copies as provisional. The agent alone is the opposite: it is too lenient to be trusted (5 wrong answers judged right). Combining them, with the agent used **only** in the uncertain band, gives the lowest error and removes 17 of the 19 provisional grades without letting a wrong answer through. A call to the agent takes 3.5 seconds at the median (17.5 s in the worst case) on the laptop. I kept the thresholds at 0.45 and 0.80: with the agent, every high threshold between 0.65 and 0.85 gave the same error, and the slightly better pair found by the sweep only won by one answer, which was not worth the risk.

These figures come from a small, hand-made benchmark and should be confirmed on real copies, but they show the main lesson of the project again: a small model is useful as a judge only behind deterministic rules and on a narrow, well-defined task.

6.6 Calibration of MMR

To choose λ , I built a controlled case that mimics a course: 4 sections, each represented by 3 overlapping chunks (vectors close to a section centre), and a query more related to the first sections than to the last ones. For each value of λ , I selected 4 passages with the same `_mmr_select` function used in production and counted how many distinct sections were covered, averaged over 200 random draws (Figure 21).

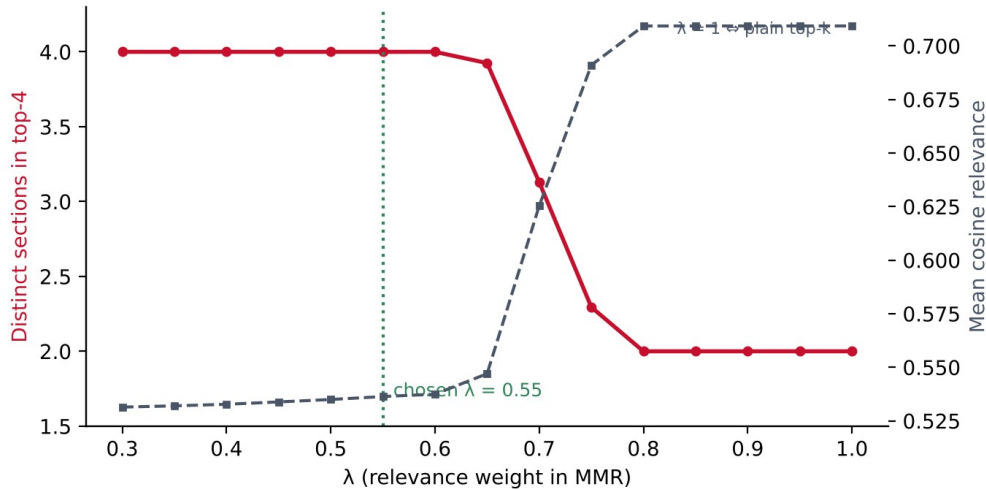


Figure 21: Sections covered by the 4 selected passages as a function of λ (synthetic course, 200 draws)

With $\lambda \geq 0.8$, MMR behaves like plain top-k and covers only 2 of the 4 sections: the quiz would focus on two parts of the course. Coverage starts to fall above $\lambda = 0.65$, and all 4 sections are covered for $\lambda \leq 0.6$. I chose $\lambda = 0.55$, which keeps full coverage with a margin while staying as close as possible to the relevance-first behaviour. As the second curve shows, the price is a lower average relevance of the selected passages, which is acceptable here because the passages remain from the same document.

6.7 Qualitative Analysis of Generated Questions

Reading the questions was, in the end, the most useful evaluation. Table 12 shows typical examples from the stored quizzes.

Table 12: Evolution of the generated questions during the internship

Stage	Example of generated question	Assessment
Mock provider (30 July)	"Which of these statements really corresponds to the course content?" with four sentences taken from the slides	Tests recognition of a sentence, no understanding; distractors about other topics
Local model, early prompt (31 July)	"The rule base of an expert system mainly contains production rules and heuristic knowledge. What is the main purpose of these rules?"	Real conceptual question, but the source excerpt is "[PASSAGE 1]"
Local model + agent (10 Aug.)	"What is the result of $3 * 4 * 5 * 6 + 1$?" with choices 181–184 and 183 marked correct	Wrong: the correct result (361) is not among the choices, and the explanation admits it
Local model + gates (2 Oct.)	"Le raisonnement procédural implique... A. L'utilisation de règles de production pour résoudre des problèmes"	Conceptual, in the language of the course, plausible distractors; grounded excerpt

The progression is clear, but some problems remain with the current version, and I prefer to state them:

- **Arithmetic and code execution errors.** The $3 * 4 * 5 * 6 + 1$ question passed every check: the excerpt is grounded and the verifier, being the same small model, did not compute the result. Questions that require calculation would need a dedicated check, for example executing simple Python expressions.
- **Missing reference answers for open questions.** In the quiz of 2 October, the two open questions have an empty reference answer (visible in the PDF export, Figure 17); overall, 6 of the 18 stored open questions are in this case. This is what led me to rework the grading: such questions are now flagged in the gradebook and graded by the agent from the course passage, or left for the teacher. Rejecting them at generation time with a new gate would be better still.
- **Some distractors are too close or partially correct.** In one question, a distractor was an extended version of the correct answer.

6.8 Toward a Quantitative Evaluation

To go beyond visual inspection, I wrote an evaluation script (`scripts/evaluate.py`) with three measures: the precision, recall and F1 of the quality gates against human labels ("good" / "bad" question), the coverage of the retrieved passages with and without MMR (pages covered and average dissimilarity between passages), and generation statistics (retention rate, share of grounded MCQs). The script also prints warnings when the labelled set is too small to be meaningful (fewer than 30 questions). The infrastructure is ready, but I did not have time to label a large enough set of questions during the internship; building this dataset is the first item of the perspectives. For the grading, a second script (`scripts/evaluate_grading.py`) already measures the error on labelled answers and can sweep the two thresholds; its results are given in Section 6.5.

Conclusion

The test suite shows that the application behaves as specified, and the measurements on real data show that the retrieval and control mechanisms I added had a measurable effect. They also show the remaining limits of a small local model, which define the next steps of the project.

General Conclusion and Perspectives

During this internship at Esprit Tech, I designed and developed QuizBot, an application that generates self-assessment quizzes from a course uploaded by a teacher, lets students take them and gives them immediate, detailed feedback. The application covers the whole scope of the specification: PDF and PPTX ingestion, a complete RAG pipeline, MCQ and open question generation, automatic grading, a student space with statistics, PDF and JSON export, authentication with roles, Docker deployment and a user guide. I added a teacher gradebook with item analysis and a three-tier grading of open answers. The whole project is covered by 150 automated tests.

Beyond the specification, I made a few choices that shaped the project. Running the language model locally, a 4-bit Qwen2.5 on a 6 GB laptop GPU, kept course content on the machine and removed any per-request cost, but it also meant working with a model that makes real mistakes. Most of my work then went into controlling quality: cleaning the input, retrieving relevant and diverse passages with a document-derived query and MMR, writing explicit rules and calibration examples in the prompt, salvaging truncated outputs, adding deterministic quality gates, and building an agent that verifies and regenerates weak questions. The same approach, rules first and the model only where rules cannot decide, then improved the grading of open answers. The replay of the gates, the statistics of the agent and the grading benchmark show that these mechanisms had a concrete effect, and also where they stop.

On a personal level, this internship taught me several things that courses alone do not. The most important is probably that with LLMs, the hard part is not getting an output but knowing whether it is good, and that measuring (even with simple tools) is better than judging by eye. I also learned to read a model's failures as information: almost every gate and every test in the project started as a strange question I did not expect. Finally, I improved my skills in software engineering around machine learning: API design, security, testing with mocks, and deployment.

Perspectives

- **Build a labelled evaluation set** of 50 or more real generated questions and use the existing script to measure the precision and recall of the gates and of the verifier.
- **Add the missing gates** identified in Chapter 6: reject open questions without a reference answer at generation time, and execute simple arithmetic or Python expressions to check answers that can be computed.
- **Let the teacher edit** a single question or regenerate it individually from the preview, instead of regenerating the whole quiz.
- **Validate the grading on real copies.** The three-tier grading and its agent were measured on 48 hand-written answers; the next step is to compare them with a teacher's grades on real student copies, and to let the teacher confirm provisional grades directly from the gradebook.
- **Support scanned PDFs** with OCR (for example Tesseract) and extract text from images and formulas, since some courses, like the machine learning slides, contain little text.

- **Move documents, quizzes and results to the relational database** (already in place for users) and process large documents asynchronously with a task queue.
- **Extend the teacher dashboard** to follow a class over several quizzes and over time, combining the gradebook and the semantic map.
- **Fine-tune a small model** on the questions validated by teachers, once enough of them are collected.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [2] Qwen Team, "Qwen2.5 Technical Report," arXiv:2412.15115, 2024.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in NeurIPS, 2020.
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of EMNLP-IJCNLP, 2019.
- [5] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in NeurIPS, 2020.
- [6] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, 2020.
- [7] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in Proceedings of ACM SIGIR, 1998.
- [8] T. Detrmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in NeurIPS, 2023.
- [9] A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," in NeurIPS, 2023.
- [10] L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in NeurIPS Datasets and Benchmarks Track, 2023.
- [11] G. Kurdi, J. Leo, B. Parsia, U. Sattler and S. Al-Emari, "A Systematic Review of Automatic Question Generation for Educational Purposes," International Journal of Artificial Intelligence in Education, vol. 30, pp. 121–204, 2020.
- [12] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," Journal of Machine Learning Research, vol. 9, pp. 2579–2605, 2008.
- [13] FastAPI documentation, <https://fastapi.tiangolo.com>
- [14] Chroma documentation, <https://docs.trychroma.com>
- [15] Hugging Face Transformers and bitsandbytes documentation, <https://huggingface.co/docs>
- [16] Streamlit documentation, <https://docs.streamlit.io>
- [17] R. L. Ebel, "Procedures for the Analysis of Classroom Tests," Educational and Psychological Measurement, vol. 14, no. 2, pp. 352–364, 1954.

- [18] L. J. Cronbach, "Coefficient Alpha and the Internal Structure of Tests," *Psychometrika*, vol. 16, no. 3, pp. 297–334, 1951.

Appendix A: Main Configuration Variables

Variable	Default	Role
LLM_PROVIDER	mock	openai, mistral, huggingface, local or mock
LOCAL_LLM_MODEL	Qwen/Qwen2.5-7B-Instruct	Local model loaded on the GPU
LOCAL_LLM_FALLBACK_MODEL	Qwen/Qwen2.5-3B-Instruct	Loaded if the main model does not fit
LOCAL_LLM_LOAD_IN_4BIT	true	4-bit NF4 quantization
LOCAL_LLM_MAX_NEW_TOKENS	3600	Minimum generation budget
LOCAL_LLM_TOKENS_PER_QUESTION	420	Budget added per requested question
EMBEDDING_MODEL	all-MiniLM-L6-v2	sentence-transformers model
CHUNK_SIZE_TOKENS / CHUNK_OVERLAP_TOKENS	800 / 150	Chunking parameters
OPEN_ANSWER_LOW / OPEN_ANSWER_HIGH	0.45 / 0.80	Similarity below which an open answer is wrong / from which it is right
OPEN_ANSWER_AGENT / ..._MAX_CALLS	true / 6	Grading agent for the uncertain band; maximum calls per copy
JWT_SECRET_KEY / ACCESS_TOKEN_EXPIRE_MINUTES	(dev value) / 480	Token signature and lifetime
PROFESSOR_SIGNUP_CODE	(empty)	Code required to register as a teacher
DATABASE_URL	sqlite:///data/quizbot.db	User database (PostgreSQL possible)

Appendix B: Project Structure

```

quizbot/
├── backend/
│   ├── main.py           # FastAPI application and endpoints
│   ├── auth.py           # bcrypt, JWT, role-based dependencies
│   ├── config.py         # central configuration (.env)
│   ├── database.py, db_models.py # SQLAlchemy engine and users table
│   ├── models.py         # Pydantic schemas
│   ├── ingestion.py      # extraction, cleaning, chunking
│   ├── embeddings.py     # sentence-transformers wrapper
│   ├── vectorstore.py    # ChromaDB wrapper
│   ├── llm_client.py     # OpenAI / Mistral / HF / local / mock providers
│   ├── quiz_generator.py # retrieval, MMR, prompt, parsing, quality gates
│   ├── quiz_agent.py     # generate - verify - regenerate loop
│   ├── grading.py        # MCQ and three-tier open-answer grading
│   ├── grading_agent.py  # LLM judge for uncertain open answers
│   ├── analytics.py      # gradebook and item analysis
│   ├── semantic_map.py   # 2D projection and performance overlay
│   ├── export.py         # PDF (ReportLab) and JSON export
│   └── storage.py        # JSON persistence of documents, quizzes, results
├── frontend/             # Streamlit app, gradebook view, hero, semantic map view
├── scripts/              # evaluate.py, evaluate_grading.py + benchmark, migration
├── tests/                # 150 pytest tests + PDF fixture
├── Dockerfile.backend, Dockerfile.frontend, docker-compose.yml
├── README.md, GUIDE_UTILISATION.md
└── requirements.txt

```